



# **6CCS3PRJ Final Year Logic Gate Circuits: A Visual Computation**

Final Project Report

Author: Shreeya Chandel

Supervisor: Dr. Josh Murphy

Student ID: 22018996

April 17, 2025

## **Abstract**

Logic gate circuits are representations of complex flows of binary logic, with particular emphasis on their interactions. This project introduces a web-based platform- LogiDex, designed for the creation, visualisation, manipulation and computation of such circuits. By providing an intuitive and interactive tool, LogiDex aims to enhance the understanding of digital logic through a visual approach to circuit creation and computation. The platform features four main features- a dynamic canvas with flexible manipulation of circuits, a formula-to-visual model converter, truth table generation and querying and propositional reduction to normal forms. With a focus on education and efficient computation, this platform aims to be an effective educational tool for novice users, as well as an informative solution for large logic problems.

### **Originality Avowal**

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Shreeya Chandel

April 17, 2025

## **Acknowledgements**

I express my sincere gratitude to my project supervisor, Dr. Josh Murphy, for his guidance and direction through each iteration of this project. His insightful feedback alongside the knowledge provided by the Computational Models module he taught was instrumental to the development and success of this project. I also extend my appreciation to my classmates, whose continuous user feedback during the development phase of the product directed me towards refining and improving my work. I am fortunate to have the support and assistance of everyone involved.

# Contents

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>3</b>  |
| 1.1      | Motivation . . . . .                                | 3         |
| 1.2      | Scope and Objectives . . . . .                      | 4         |
| <b>2</b> | <b>Background and Context</b>                       | <b>5</b>  |
| 2.1      | Brief . . . . .                                     | 5         |
| 2.2      | Boolean Algebra and Logic Gates . . . . .           | 5         |
| 2.3      | Propositional Logic and Symbols . . . . .           | 6         |
| 2.4      | Laws of Boolean Algebra . . . . .                   | 6         |
| 2.5      | Conjunctive and Disjunctive Normal Forms . . . . .  | 7         |
| 2.6      | Abstract Syntax Trees . . . . .                     | 8         |
| 2.7      | Shunting Yard Algorithm . . . . .                   | 8         |
| <b>3</b> | <b>Literature Review</b>                            | <b>9</b>  |
| 3.1      | Brief . . . . .                                     | 9         |
| 3.2      | Review of Existing Work . . . . .                   | 9         |
| 3.3      | Extending Existing Tools: A Comparison . . . . .    | 11        |
| <b>4</b> | <b>Requirements and Specification</b>               | <b>14</b> |
| 4.1      | Brief . . . . .                                     | 14        |
| 4.2      | Functional Requirements . . . . .                   | 14        |
| 4.3      | Non-Functional Requirements . . . . .               | 17        |
| 4.4      | Specification . . . . .                             | 18        |
| <b>5</b> | <b>Design</b>                                       | <b>20</b> |
| 5.1      | Brief . . . . .                                     | 20        |
| 5.2      | System Architecture . . . . .                       | 20        |
| 5.3      | Use Cases . . . . .                                 | 21        |
| 5.4      | API Flow . . . . .                                  | 23        |
| 5.5      | Graphical User Interface . . . . .                  | 29        |
| <b>6</b> | <b>Implementation</b>                               | <b>36</b> |
| 6.1      | Brief . . . . .                                     | 36        |
| 6.2      | Code Structure . . . . .                            | 36        |
| 6.3      | Canvas Creation and Dynamic Visualisation . . . . . | 37        |
| 6.4      | Computational Section – Validation . . . . .        | 43        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 6.5      | Truth Table Generation . . . . .                      | 44        |
| 6.6      | Propositional Reduction Tool . . . . .                | 46        |
| 6.7      | Formula-to-Circuit Converter . . . . .                | 52        |
| <b>7</b> | <b>Testing and Evaluation</b>                         | <b>56</b> |
| 7.1      | Brief . . . . .                                       | 56        |
| 7.2      | Automated Unit Testing . . . . .                      | 56        |
| 7.3      | User Testing and Manual Testing . . . . .             | 57        |
| 7.4      | Evaluation . . . . .                                  | 57        |
| <b>8</b> | <b>Legal, Social, Ethical and Professional Issues</b> | <b>60</b> |
| <b>9</b> | <b>Conclusion and Future Work</b>                     | <b>62</b> |
| 9.1      | Conclusions . . . . .                                 | 62        |
| 9.2      | Future Work . . . . .                                 | 63        |
|          | Bibliography . . . . .                                | 65        |
| <b>A</b> | <b>User Guide</b>                                     | <b>66</b> |
| A.1      | ReadME File: Instructions to Run the Code . . . . .   | 73        |

# Chapter 1

## Introduction

The aim of this project is to build a versatile tool to visualize logic gates, construct logic diagrams, and allow for step-by-step computation. The main priority is to design the project for a diverse audience, with multiple tiers of usability ranging from beginners exploring binary logic to experienced users seeking accurate and efficient computational assistance, enhanced by visual indicators for clarity.

This tool will provide users with an interactive experience to facilitate a deeper understanding of binary logic in a practical setting with learning-focused benefits. Emphasis has been placed on visual components and dynamic feedback in the design, allowing users to grasp the connections between various logic gates and how they form complex circuits. Features such as dynamic toggling of input values, error-specific feedback, drag-and-drop adjustments, and output-guided insights, formula-to-circuit responses will provide a hands-on learning experience, making abstract concepts more tangible.

### 1.1 Motivation

As an Informatics student, I use binary logic for many projects and applications. In the initial phase of learning, as well as eventually for larger computations, I struggled to understand and visualise the behavior of logic gates and complex circuits without extensive error-prone calculations. The abstract and formulaic nature of propositional logic makes it difficult to interpret or apply instinctively. To investigate the commonality of this challenge, I conducted primary research among fellow students who were also studying similar modules.

The outcome of the research was consistent with my experience: many students similarly

reported that binary logic beyond simple structures became confusing when presented in textual or formulaic formats. In the secondary research phase, I realised that existing tools in this domain lacked flexibility, instruction, clarity and sophistication.

The motivation for creating **LogiDex** was to create a user-friendly tool that makes logic gates tangible and clear through visual interaction. I was motivated to develop an interface that was designed iteratively based on feedback from the target cohort, ensuring that the needs I recognised during research were actually met throughout the development process.

## 1.2 Scope and Objectives

The scope of this project is centered around developing a dynamic, interactive program with a focus on visualisation and analysis. The system is design around 3 core themes: experimental and interactive freedom, educational value and accessibility for a wide range of users - from beginners to advanced users.

The scope of the project is described as follows.

- **Representation of logic circuits:** The system allows users to visually construct and understand logic circuits using standard gates like AND, OR, NOT, XOR, etc.
- **Interaction of binary values with gates:** Users can toggle input values to see real-time feedback in the circuit's output, offering insights into gate behavior and logic flow.
- **Propositional logic processing:** Users can input logical expressions using propositional symbols, which are parsed into circuits using abstract syntax trees (ASTs) and reduced to canonical forms (CNF/DNF).
- **Truth tables and data querying:** The system generates truth tables from circuits and formulas, allowing users to explore all possible input-output mappings and verify logical correctness.
- **Fast, reactive experience:** The tool is built for responsiveness — offering seamless updates and dynamic feedback during interaction, ensuring smooth user experience.

## Chapter 2

# Background and Context

### 2.1 Brief

This chapter introduces concepts regarding the underlying theory that is the basis of this project. The domain of propositional logic is explored in a contextual manner. Boolean algebra, normal forms and propositional symbols have been introduced. Abstract Syntax Trees, Breadth-First Sorting, and related topics are explored as well.

### 2.2 Boolean Algebra and Logic Gates

Binary logic is a system in which variables assume one of two possible values: 0 (false) or 1 (true). This foundational model enables the design of digital logic circuits used in computing hardware. Boolean algebra, introduced by George Boole in the 19th century, provides a formal framework to describe binary operations [8].

Logic gates are the building blocks of digital circuits. There are seven fundamental gates:

- AND ( $A \wedge B$ ) – Outputs true only when both inputs are true.
- OR ( $A \vee B$ ) – Outputs true when at least one input is true.
- NOT ( $\neg A$ ) – Inverts the input.
- NAND ( $\neg(A \wedge B)$ ) – Inverse of AND.
- NOR ( $\neg(A \vee B)$ ) – Inverse of OR.
- XOR ( $A \oplus B$ ) – Outputs true when inputs differ.

- XNOR ( $\neg(A \oplus B)$ ) – Inverse of XOR.

These gates can be represented both as circuit components and propositional formulas [16]. Each gate has a corresponding truth table that describes its output for every combination of inputs.

## 2.3 Propositional Logic and Symbols

Propositional logic (also known as sentential logic) is the formal study of propositions and their logical relationships [15]. A proposition is a statement that is either true or false but not both. Symbols such as  $\wedge$  (AND),  $\vee$  (OR),  $\neg$  (NOT),  $\rightarrow$  (IMPLICATION), and  $\leftrightarrow$  (BICONDITIONAL) are used to build logical expressions from atomic variables.

These expressions serve as the basis for evaluating the behavior of digital circuits. In tools like LogiDex, each propositional symbol is mapped to a corresponding logic gate, facilitating seamless conversion between symbolic and circuit-based representations [8].

## 2.4 Laws of Boolean Algebra

Boolean algebra provides a set of identities (or laws) that are universally true for all Boolean values. These identities allow logical expressions to be simplified or transformed, much like arithmetic laws simplify numeric expressions. These laws form the basis for logic minimization and circuit equivalence checking [5, 8].

- **Idempotent Law:**

$$A \wedge A = A, \quad A \vee A = A$$

Using the same value multiple times doesn't affect the outcome.

- **Commutative Law:**

$$A \wedge B = B \wedge A, \quad A \vee B = B \vee A$$

The order of operands doesn't matter for conjunction or disjunction.

- **Associative Law:**

$$A \wedge (B \wedge C) = (A \wedge B) \wedge C, \quad A \vee (B \vee C) = (A \vee B) \vee C$$

Grouping of operations with the same operator is irrelevant.

- **Distributive Law:**

$$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$$

$$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$$

One operator distributes over the other.

- **Identity Law:**

$$A \wedge TRUE = A, \quad A \vee FALSE = A$$

TRUE (1) and FALSE (0) act as identity elements for AND and OR respectively.

- **Complement Law:**

$$A \wedge \neg A = FALSE, \quad A \vee \neg A = TRUE$$

A value combined with its logical negation always yields a constant.

- **De Morgan's Laws [5]:**

$$\neg(A \wedge B) = \neg A \vee \neg B$$

$$\neg(A \vee B) = \neg A \wedge \neg B$$

These laws show how to distribute negation over AND/OR, and they are critical for transforming logic expressions and circuit design.

These fundamental identities are taught in introductory courses on digital logic and formal reasoning [5]. They are often proven via truth tables and play a central role in converting circuits to canonical forms or simpler equivalents. For instance, De Morgan's laws are vital when transforming NAND-only circuits into ones using OR and NOT gates, which is particularly useful in hardware logic simplification.

## 2.5 Conjunctive and Disjunctive Normal Forms

To systematically manipulate propositional logic expressions, they are often transformed into standard forms:

- **Conjunctive Normal Form (CNF):** A conjunction of disjunctions (e.g.,  $(A \vee B) \wedge (\neg C \vee D)$ )

- **Disjunctive Normal Form (DNF):** A disjunction of conjunctions (e.g.,  $(A \wedge B) \vee (\neg C \wedge D)$ )

These normal forms are particularly useful in applications like logic circuit design, SAT solvers, and formal verification [3]. Transformation into CNF and DNF enables a standardized way of evaluating logical equivalences and conducting symbolic computation.

## 2.6 Abstract Syntax Trees

An Abstract Syntax Tree (AST) is a tree-based representation of the syntactic structure of a formula or expression [9]. Each internal node represents an operator (e.g.,  $\wedge$ ,  $\vee$ ,  $\neg$ ), and each leaf node represents an operand or variable. For example, the formula  $A \wedge (B \vee C)$  would be represented as a binary tree with  $\wedge$  at the root and  $\vee$  as a child node.

ASTs serve as intermediate representations during parsing and transformation. They enable systematic traversal and manipulation of formulas and are heavily used in compilers and symbolic logic tools [17].

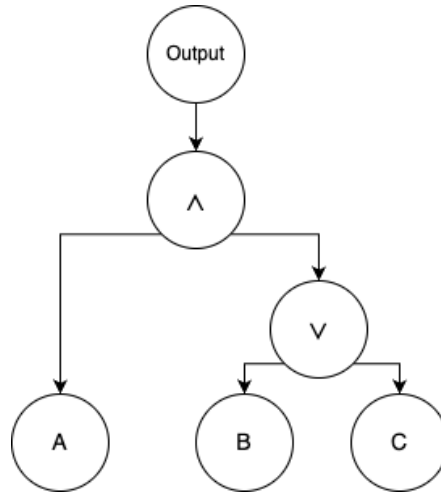


Figure 2.1: Example of an AST for the expression  $\neg A \wedge (B \vee C)$

## 2.7 Shunting Yard Algorithm

The Shunting Yard algorithm, proposed by Edsger W. Dijkstra, is used to parse infix expressions (like  $A \wedge B$ ) into postfix notation (also known as Reverse Polish Notation) [6]. This is particularly useful for building ASTs because postfix notation does not require parentheses and allows for efficient left-to-right parsing.

## Chapter 3

# Literature Review

### 3.1 Brief

This chapter focuses on exploring the current state of the field of study the program is in. During the process of formulating an ideal approach for the project, several established models were studied and referenced. The goal was to retain the strengths of these tools while addressing their limitations and introducing novel features that enhance both functionality and user-friendliness. The subsequent sections give us an insight into related literature, existing tools, and their functionality. They also highlight how the development process of LogiDex aims to extend these functionalities and also overcome the limitations other functionalities provide.

### 3.2 Review of Existing Work

- **Static Grid GUI:** Existing tools rely on static grids for adding and removing gates, which restricts circuit expansiveness and limits user creativity. Static GUIs often feel rigid, particularly for larger circuits.

**Improvement:** Our tool employs a dynamic drag-and-drop GUI with fluid wire connections, providing a seamless and intuitive experience. Users can build expansive circuits with minimal constraints.

- **Drag-and-Drop Functionality:** *ScienceDemos.org.uk* [12] and *Simulator.io* [13] offer drag-and-drop interfaces for logic gates but lack real-time feedback or error correction. These tools allow users to create any format and combination of elements and notes, with any number of wires and combinations, without providing guidance or correction. This

results in halted computations without feedback, leaving users frustrated or confused.

**Improvement:** Real-time error detection and feedback are introduced in our tool to prevent invalid computation, enhance learning, and strike a balance between flexibility and structured diagram creation.

- **Binary Value Indication:** Tools like *Logicly* [10] or *CircuitVerse* [4] either lack real-time colour coding or use unintuitive schemes (e.g., blue-white) that do not clearly convey logical states.

**Improvement:** A more intuitive colour scheme with real-time updates (e.g., green for true, red for false) is implemented to make binary logic states visually clear and easy to understand.

- **Truth Table Generation:** Most tools do not automatically generate truth tables, leaving users to compute them manually.

**Improvement:** Automatic truth table generation is integrated into our tool, offering users a quick and accurate overview of input-output mappings for their circuits.

- **Educational Support:** Pre-existing tools lack functionality for computational breakdown and guided learning. Their sole focus on functionality neglects users who may be unfamiliar with binary logic.

**Improvement:** Our tool integrates hover-based explanations for each gate, enables querying of individual computation steps, and includes a guided learning mode where users can input the next step. This offers a comprehensive learning experience tailored for both beginners and advanced users.

### 3.3 Extending Existing Tools: A Comparison

Table 3.1: Comparison of LogiDex with Existing Logic Circuit Tools Based on Unique Features

| Criteria                             | LogiDex                                                                                                                                                     | Logicly [10]                                                                | CircuitVerse [4]                                                              | Academo [2]                                           |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Formula-to-Circuit Generation</b> | <b>Only tool that automatically generates a complete circuit from a user-entered propositional formula. Fully interactive and editable post-generation.</b> | No formula interpretation or conversion. Circuits must be manually built.   | Manual circuit creation only. No formula-based circuit synthesis.             | Not supported. Limited manual circuit design only.    |
| <b>Truth Table Presentation</b>      | <b>Automatically generated with logical filtering, clear visual layout, and input-output highlight. Presented in an elegant, accessible format.</b>         | Supports exporting truth tables, but lacks visual integration or filtering. | Truth table available but not styled or filtered. Limited to flat CSV format. | Truth tables not supported. Purely visual simulation. |

*Continued on next page*

| Criteria                               | LogiDex                                                                                                                                    | Logically [10]                                                                                     | CircuitVerse [4]                                                                    | Academio [2]                                                            |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Step-by-Step Logic Reduction</b>    | Unique: Symbolic formula simplification (CNF/DNF) shown step-by-step with explanations. Each transformation is numbered and visual.        | Not supported. No symbolic logic features.                                                         | No support for symbolic manipulation or reductions.                                 | No support for formulaic logic or reduction.                            |
| <b>Node Interactivity and Feedback</b> | Interactive nodes with real-time value coloring (green/red), visual signal propagation, and click-based computation queries at every node. | Limited visual feedback. Coloring less intuitive. Nodes are not interactive beyond basic toggling. | Simulation with some visual feedback, but querying or flow visualization is absent. | Very minimal interaction. Gate values not visually traced or queryable. |
| <b>Error Handling and Guidance</b>     | Errors are shown contextually with detailed prompts and suggestions for correction. Guides the user toward a fix rather than halting.      | No error messages beyond basic blocking. No user guidance or feedback.                             | Some warnings, but minimal help. Users left to debug visually.                      | No error detection or messaging. Malfunctions go undi-<br>cated.        |

*Continued on next page*

| Criteria                              | LogiDex                                                                                                                     | Logicly [10]                                                        | CircuitVerse [4]                                                                  | Academo [2]                                                            |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Circuit Editor and Flexibility</b> | <b>Fluid drag-and-drop with responsive node rendering, custom edge drawing, and free-form canvas. Intuitive and modern.</b> | Standard drag-and-drop. Some flexibility but lacks modern fluidity. | Supports subcircuits and collaboration but can become rigid with complex layouts. | Extremely basic. Static layout. No wire customization or dynamic flow. |

## Chapter 4

# Requirements and Specification

### 4.1 Brief

This section introduces the key requirements for the Logic Gate Circuit Visualisation project, highlighting both the functional and non-functional requirements. The requirements are numbered and organised with utility tags for future reference and callbacks in the project. Functional requirements include the requirements for user interactions as well as the system functionality.

### 4.2 Functional Requirements

#### 4.2.1 User Requirements

This section highlights the user and interface-level requirements of the program. It reflects the interactions the user must be able to make with the program and the features available to them to satisfy the user experience. The following keys represent the tags used in the table below:

- A: assistive
- C: computational
- V: visualisation
- Vld: validation

Table 4.1: User Requirements

| ID    | Requirement                                                                                                           | Tags   |
|-------|-----------------------------------------------------------------------------------------------------------------------|--------|
| UR-01 | User must be able to interact with a canvas to add, drag and remove nodes                                             | V      |
| UR-02 | User must be able to scroll and select nodes from the toolbar                                                         | V, A   |
| UR-03 | User must be able to connect nodes via wires without invalid links                                                    | V, Vld |
| UR-04 | User must be able to toggle input values between 0 and 1                                                              | C      |
| UR-05 | User must be able to see green and red color-coded logical values on nodes as the values traverse through the circuit | V      |
| UR-06 | User must be able to flip a gate node to view details and intermediate truth table explanation to query the circuit   | V, C   |
| UR-07 | User must be able to reset the canvas and clear all logic                                                             | A      |
| UR-08 | User must be able delete nodes from the canvas                                                                        | A      |
| UR-09 | User must be able to switch between Truth Table and Propositional Reduction tabs                                      | A , C  |
| UR-10 | User receives specific error messages when trying to compute an invalid circuit                                       | A, Vld |
| UR-11 | User must be able to view the complete truth table of valid circuit                                                   | V , C  |
| UR-12 | User must be able to filter rows singularly and compoundly in truth table and clear them                              | A      |
| UR-13 | User must be able to view the number of rows generated in the truth table which gets modified upon filtering          | A      |
| UR-14 | User must be able to see true/false counts for each column                                                            | V      |
| UR-15 | User can compute the propositional formula of the circuit diagram                                                     | C      |
| UR-16 | User can view CNF/DNF formulas and their reductions step-by-step                                                      | C      |
| UR-17 | User can input a formula and see the corresponding circuit rendered on the canvas                                     | C      |
| UR-18 | User sees specific validation errors if formula is invalid and how to correct them                                    | A, Vld |
| UR-19 | User cannot add more than 10 input nodes to the canvas                                                                | A, Vld |
| UR-20 | User is shown a welcome modal on first load                                                                           | V, A   |
| UR-21 | User can view a tutorial of the tool                                                                                  | V, A   |

### 4.2.2 System Requirements

This section outlines the technical and implementation-level requirements of the system. These requirements describe the functionality the system must support in order to satisfy the intended features and ensure reliability, interactivity, and logical correctness. The following keys represent the tags used in the table below:

- V: visualisation
- C: computational
- A: assistive
- Vld: validation

Table 4.2: System Requirements and Tags

| ID    | Requirement                                                                                                         | Tags   |
|-------|---------------------------------------------------------------------------------------------------------------------|--------|
| SR-01 | The system must render an interactive canvas with drag-and-drop node functionality                                  | V      |
| SR-02 | The system must generate unique node IDs and maintain state mapping                                                 | C      |
| SR-03 | The system must prevent invalid connections (loops, multiple outputs)                                               | C, A   |
| SR-04 | The system must provide a toolbar for selecting gates and nodes                                                     | V, A   |
| SR-05 | The system must reset the UI and state when the reset button is clicked                                             | V      |
| SR-06 | The system must support binary toggle input nodes with circuit update                                               | C, V   |
| SR-07 | The system must provide real-time node coloring based on logical flow in the circuit                                | V, C   |
| SR-08 | The system must ensure node deletion removes the node and related connections                                       | A      |
| SR-09 | The system must allow nodes to reveal gate-specific logic upon flipping                                             | V, C   |
| SR-10 | The system must validate the circuit and show specific errors before allowing computation                           | A, Vld |
| SR-11 | The system must evaluate and generate a truth table or a propositional formula only when the circuit is valid       | C      |
| SR-12 | The system must highlight current selected inputs in the top row and outputs in the final column of the truth table | V      |

| ID    | Requirement                                                                                                           | Tags   |
|-------|-----------------------------------------------------------------------------------------------------------------------|--------|
| SR-13 | The system must allow filtering of truth table rows based on selected columns                                         | A, V   |
| SR-14 | The system must paginate the truth table and dynamically increase the rows on demand                                  | A      |
| SR-15 | The system must show metadata: row count, number of true/false values dynamically                                     | V, C   |
| SR-16 | The system must derive a propositional formula from the circuit graph                                                 | C      |
| SR-17 | The system must reduce the formula into CNF and DNF with transformation steps using an AST                            | C      |
| SR-18 | The system must parse the inputted propositional formula into a circuit canvas                                        | C, V   |
| SR-19 | The system must validate and provide feedback for incorrect formulas                                                  | A, Vld |
| SR-20 | The system must disable addition after a maximum of 10 input nodes                                                    | Vld, A |
| SR-21 | The system must display a welcome modal with tutorial option on load                                                  | A      |
| SR-22 | The system must implement a step-by-step tutorial exploring each component in the UI and explaining its functionality | A      |

### 4.3 Non-Functional Requirements

This section describes the quality the system must provide in terms of performance, efficiency and accuracy.

The following keys represent the tags used in the table below:

- **performance** – speed and response time of the system
- **scalability** – ability to handle increasing circuit complexity
- **usability** – user experience and clarity of messages
- **design consistency** – predictable and readable layouts
- **accessibility** – compliance with accessibility standards
- **responsiveness** – compatibility with different screen sizes
- **memory session** – ability to preserve temporary states locally
- **maintainability** – modular structure and ease of testing
- **reliability** – accurate logic and error prevention

Table 4.3: Non-Functional Requirements

| ID     | Requirement                                                  | Tags               |
|--------|--------------------------------------------------------------|--------------------|
| NFR-01 | System responds to interactions within 200ms                 | performance        |
| NFR-02 | System handles circuits with up to 50 nodes efficiently      | scalability        |
| NFR-03 | System displays specific and user-friendly error messages    | usability          |
| NFR-04 | Node layout remains consistent and readable                  | design consistency |
| NFR-05 | System follows accessibility guidelines                      | accessibility      |
| NFR-06 | System is responsive across screen sizes                     | responsiveness     |
| NFR-07 | Local data (e.g., state of the tab) is preserved temporarily | memory session     |
| NFR-08 | System is modular and easily testable for extensions         | maintainability    |
| NFR-09 | System protects against circular logic evaluation            | reliability        |
| NFR-10 | Tutorial and UI render within 1 second of load               | performance        |

## 4.4 Specification

The core technologies used in the project were selected after research and comparison with alternate languages and libraries, based on the requirements of the project. The following section provides justification for the selected frameworks.

### 4.4.1 Frontend

#### React vs. Naive JavaScript

React was chosen for front-end development due to its component-lifecycle and state management, which simplifies creating an interactive and maintainable GUI. Additionally, the `react-flow` library served as a central tool for dynamic responses. In contrast, vanilla Javascript would not supply easy integration tooltip libraries, drag and drop functionality or accurate error guidance. React supports reusable components and a virtual DOM, making it ideal for features like dynamic circuit visualization and real-time feedback as well as managing interactive user input for immediate manipulation of truth tables and formulas.

React’s `useEffect` and `useRef` hooks were used extensively to manage side effects and state management. Additionally, React uses JSX — allowing the writing of HTML style structures which enabled creation of self-sufficient components, inevitably making the readability of the

code more intuitive. Pure HTML and JavaScript apps would be more rigid and error-prone, and may cause issues in adapting to the desired dynamic functionality required for this system.

#### **4.4.2 Backend**

For the backend, Flask was chosen, which is a web framework for Python. This was done as it allows seamless connection with the frontend by providing a lightweight mechanism to build RESTful APIs and manage data flows. Additionally, Python's recursion library and dynamic typing was beneficial in creation of abstract syntax trees. In contrast, Java lacks equally robust equivalent libraries which makes AST creation and parsing tougher and implementing recursive language more inefficient. As a result, Java was discarded in competition to Python (Flask) for this project.

#### **4.4.3 RESTful APIs**

For stateless communication between the frontend and backend, the application uses RESTful APIs. Flask allows for ease in request handling and routing using API calls. The API endpoints were useful for calls to large truth tables and returning JSON objects, enabling seamless exchange of data with the frontend application. Additionally, as the backend was used for creating abstract syntax trees, the propositional formulas and all the steps could be easily communicated via RESTful APIs. They provided proper error handling at the backend level with validation checks at the frontend level. They would also subsequently allow for scalability to complex operations without making any significant changes to the API structure and architecture.

# Chapter 5

## Design

### 5.1 Brief

This chapter aims to provide a perspective into the design approach and patterns in the project. Various widely accepted design diagrams have been used to illustrate different abstractions of the system design. Further, justifications for design choices have been given. To conclude, elements of the GUI have been provided and explained to provide insight.

### 5.2 System Architecture

The product created involves only two layers of architecture: the React App in the frontend and the Flask (Python) app in the backend. The project is mainly divided into two central objectives: one, the visual feedback and two, computational assistance. Each layer primarily took care of one of these objectives. The frond-end layer contains the visual components, dynamic rendering of canvas and GUI and creates the interface for user interaction. The Flask layer contains the core functionality for the computation of the binary logic interactions. Figure 5.1 shows an abstract representation of this structure.

A database layer was initially considered as well as implemented, specifically for truth table querying, but later rejected as a design choice as it failed to provide the flexibility with the overall objective of the program. The program was purposed to be a tool for flexible querying and manipulation. Querying a database was considered as it would add an extra layer of storage, however since the tool was created with the purpose of dynamic rendering, the querying and filtering was handled at the frontend level itself.

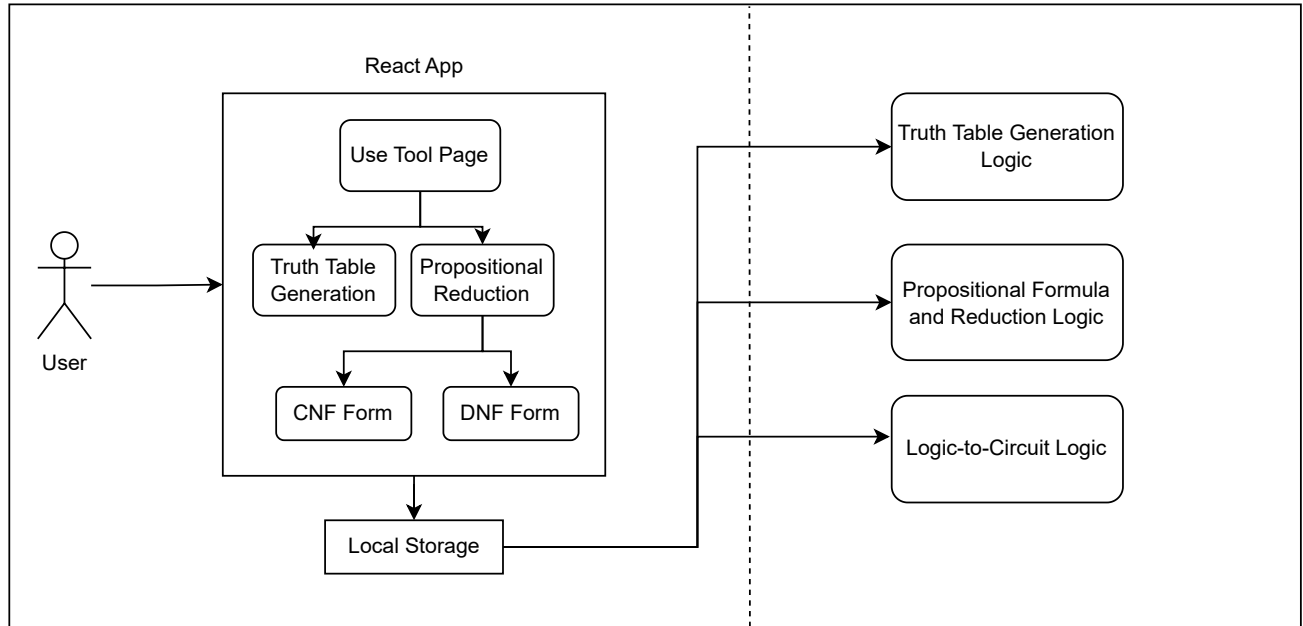


Figure 5.1: System Architecture Diagram

### 5.3 Use Cases

Use-case diagrams describe the high-level functions and scope of a system. These diagrams also identify the interactions between the system and its actors. The use cases and actors in use-case diagrams describe what the system does and how the actors use it, but not how the system operates internally.[? ]

The diagram in Figure 5.2 represents the use case diagram for LogiDex. It highlights the manner in which the user interacts with the system, the sequence of events and how they are triggered, along with the external libraries used and the points at which they assist in satisfying those interactions. As shown in the diagram, the user can either visualise a circuit by creating it on the canvas- by dragging and dropping nodes, and creating connections; or by entering a propositional formula and allowing the system to create the circuit diagram for them.

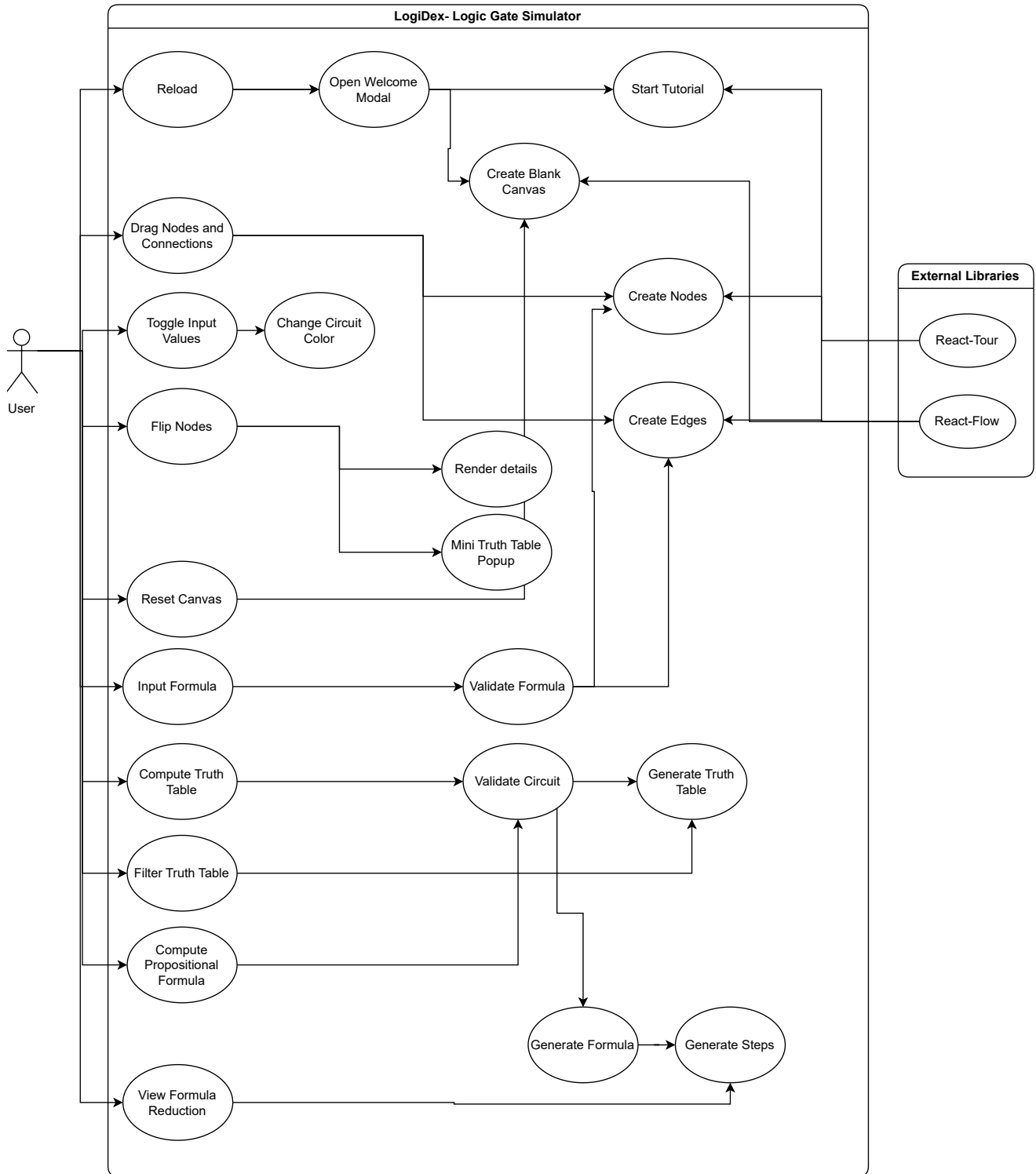


Figure 5.2: Use Case Diagram: Actor and Interactions

## 5.4 API Flow

For communication between the frontend and the backend, distinct API `POST` calls were made for the system's core features:

- `/api/compute-truth-table` – returns the generated truth table along with evaluated outputs.
- `/api/compute-propositional-formula` – returns the derived propositional formula and a step-by-step reduction to normal forms.
- `/api/create-circuit` – returns circuit data, including node and edge configuration, to be imposed onto the canvas.

The API flow graphs given below illustrate the flow of data and control between the layers for each of these respective functionalities.

### Formula to Circuit

As illustrated in Figure 5.3, the backend parses the string and extracts nodes, input values, and connections that should be made in circuit form. The frontend receives this data, either in the form of error strings, or in the form of circuit data. If error strings are found, it renders them below the input box. Each error string describes the reason for the input string being rejected, and corrects the user to the correct format. In the situation that the data fetched from the API is a JSON file containing nodes and connections, the format of data is as follows:

Listing 5.1: Sample JSON response from `/api/create-circuit`

```
{
  "nodes": [
    {"id": "A", "type": "INPUT", "inputValues": [1]},
    {"id": "B", "type": "INPUT", "inputValues": [0]},
    ...
  ],
  "connections": [
    {"source": "A", "target": "AND1"},
    ...
  ]
}
```

Note: This is an example and the data varies based on the formula received.  
This is then handled in the frontend using react-flow and helper functions.

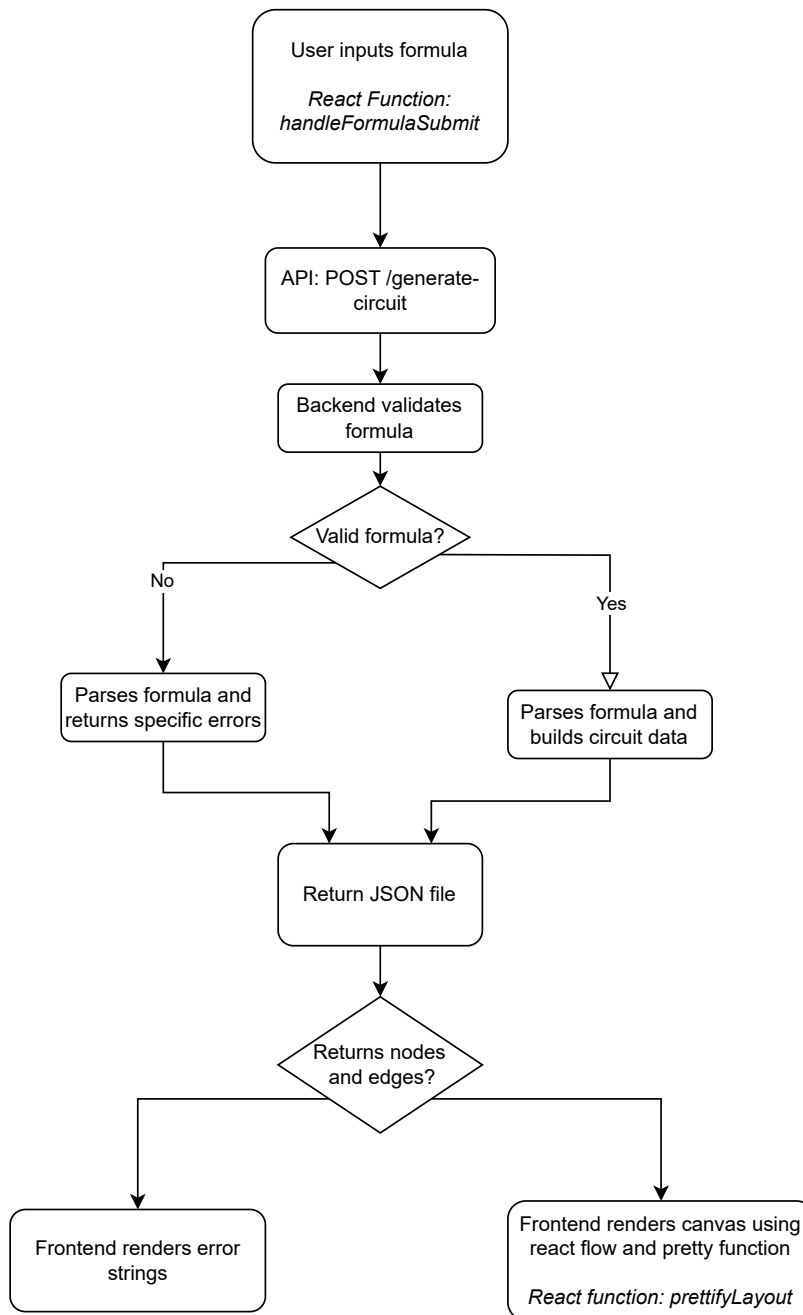


Figure 5.3: API Flow Diagram: Formula-to-Circuit Creation

## Truth Table Generation and Propositional Reduction

Due to a common validation function, the API flow for these two functionalities is very similar, with the only difference being their components and JSON outputs. The flow can be described in two parts- the user clicks a compute button and the frontend validates the system, and calls the API only if valid.

For generating truth tables, the API call returns a JSON in the following format:

Listing 5.2: Sample JSON response from `/api/compute-truth-table`

```
{
  "inputs": ["A", "B"],
  "table": [
    {
      "row": [1, 0, 0],
      "highlighted": true
    },
    {
      "row": [0, 0, 0]
    },
    {
      "row": [0, 1, 0]
    },
    {
      "row": [1, 1, 1]
    }
  ]
}
```

(Diagram on next page.)

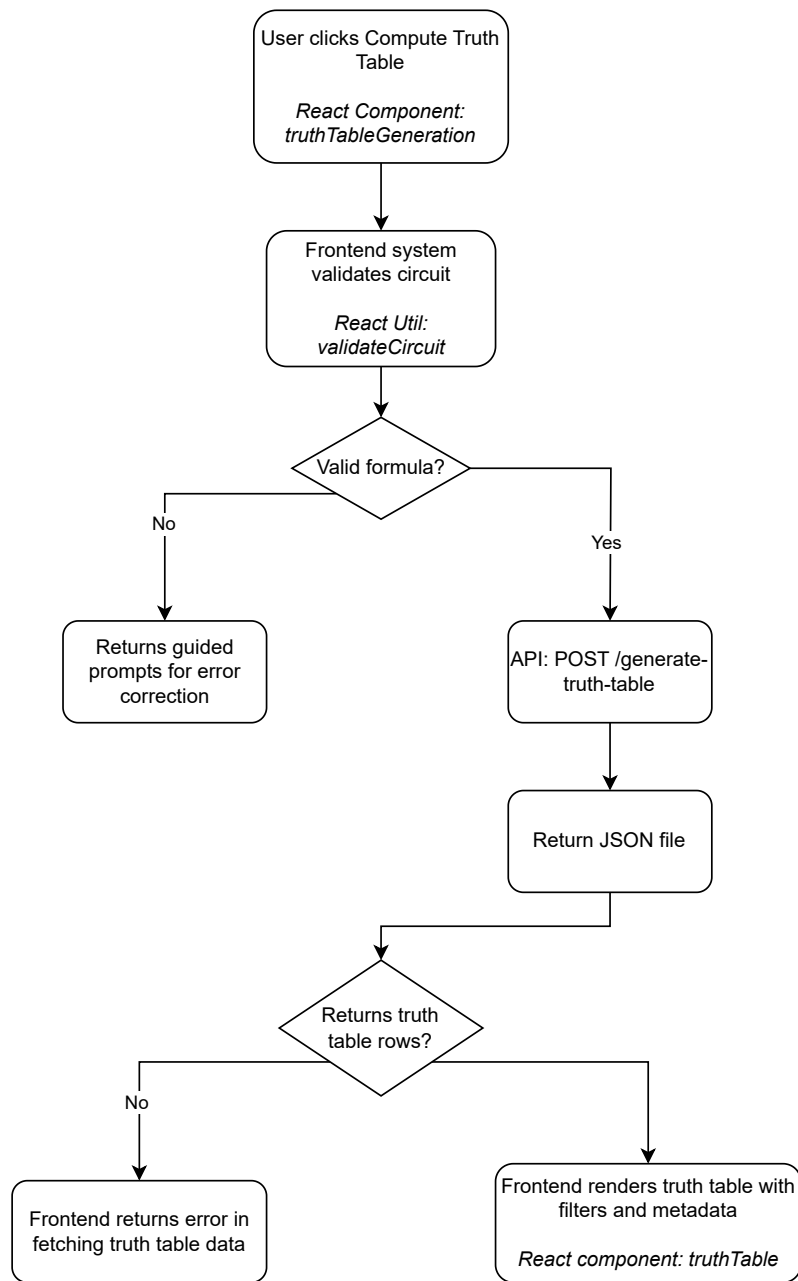


Figure 5.4: API Flow Diagram: Generating Truth Table

For generating propositional formulas, the API call fetches a JSON in the following format:

Listing 5.3: Sample JSON response for formula NOT(A OR B)

```
{
  "formula": "NOT(A OR B)",
  "cnfformula": "(NOT A OR NOT B)",
  "dnfformula": "(NOT A AND NOT B)",
  "cnfsteps": [
    {
      "step_number": 1,
      "step": "Apply De Morgan's Law to negated OR: NOT(A OR B) -> (NOT A AND NOT B)",
      "formula": "(NOT A AND NOT B)"
    }
  ],
  "dnfsteps": [
    {
      "step_number": 1,
      "step": "Apply De Morgan's Law to negated OR: NOT(A OR B) -> (NOT A AND NOT B)",
      "formula": "(NOT A AND NOT B)"
    }
  ]
}
```

(Diagram on next page.)

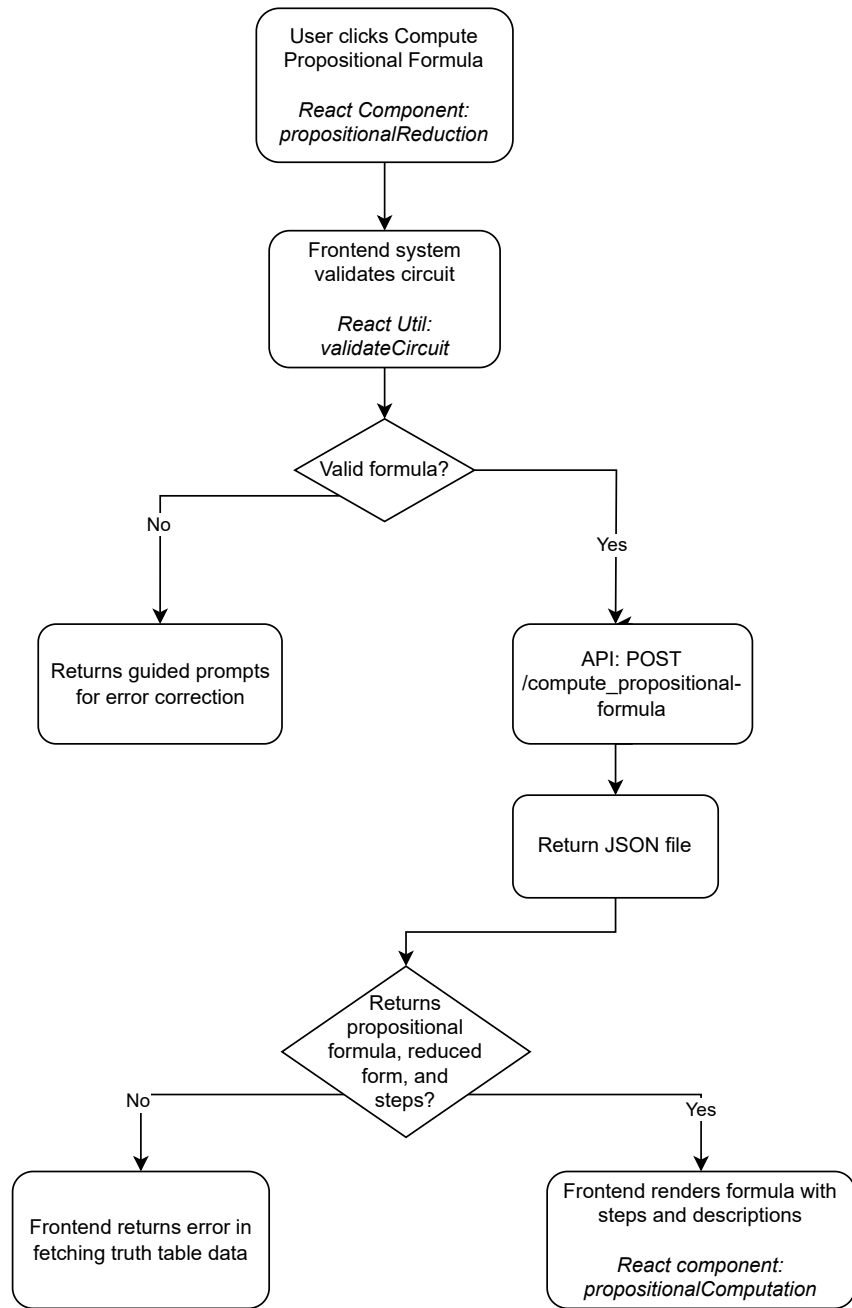


Figure 5.5: API Flow Diagram: Generating Propositional Formula and Reduction to CNF/DNF

## 5.5 Graphical User Interface

The program’s graphical user interface uses React and React-flow, with JSX for consolidating independent components. The design aims to allow interaction with complex logical circuits in a visual and intuitive way, with no constraints on exploration. This section briefs the graphical interface design decisions, with explanations for why certain elements and libraries were chosen and measures that were implemented to make the user experience clear, usable and reliable.

### 5.5.1 User Interface and Styling

The design philosophy for this project has a heavy focus on simplicity, intuition and utility. Neither is the tool meant to be professional nor was the goal to gameify it. The main objective was to maintain a clean interface and avoid unnecessary clutter without compromising on usability and features.

As an example, a previous design decision was to have a data summary next to the truth table that provides the rows generated and the number of true-false outputs per variable. While this was initially implemented, eventually, the design was modified to subtly integrate all this information into the table itself, thus allowing users to access relevant insights without distraction.

To preserve uniformity in styling and design choices, UI wire-frames were made to structure each component in the web application. Across the application, for similar components, a common set of styling files defining the color schemes and reusable CSS classes was used. This provided a cohesive appearance to the application.

### 5.5.2 Consistency

Maintaining consistency across different parts of the system was a key focus. Several techniques and checks were employed:

- For consistent visualisation in a tree-like circuit format, a `prettifyLayout` function is implemented which leads to the circuit being created like a tree when a propositional formula is inputted.
- Input node labels followed a consistent A–J pattern. When a node is deleted and re-added, the label ordering is preserved. The same notation is preserved in the truth table as well.

- Truth tables and propositional reductions shared common validation checks and styling files, as well as same error formats and strings.

### 5.5.3 Compactness and Clarity

The web application only has a single page where all functionality is present. This was a deliberate design choice. Initially, a landing page was present, with 3 buttons - Draw Canvas, View Propositional Logic, Generate Canvas from Formula. React routing was used to route between these pages.

For this purpose, reusable components were also created. The canvas element, the truth table and other components were modular and designed to be present in multiple pages. However, this interfered with the philosophy and intention of wanting the conversion to be real-time and convertible into different forms.

Therefore, the landing page and other pages were consolidated into single page, and a Welcome Modal was added to replace the utility of the landing page, which introduces the tool without extensive navigation.

### 5.5.4 Interactive Elements and Responsiveness

The foundation of the program is interactive components to enhance user learning and engagement. For this purpose, **React Flow** was used for the canvas component which provided drag-and-drop functionality to the nodes. Additionally, the node handles and edges were customisable to fit the wire connections that were consistent with logical flow. Another React library which was used is **React Tour**, which assisted in creating a tutorial functionality, traversing each element on the page and giving instructions regarding their use and outcomes.

**MaterialUI** is an open-source React component library that implements Google's Material Design. It's comprehensive and can be used in production out of the box. [?] MaterialUI's components were leveraged and modified for consistent styling and reusability. Pre-styled components like Tooltip, Button, Typography, etc. served useful in maintaining a consistent layout for the application.

### Colour Coded Feedback

Each node (Input, Gate, Output) visually updates in real-time based on logical input values as they flow through the circuit diagram.

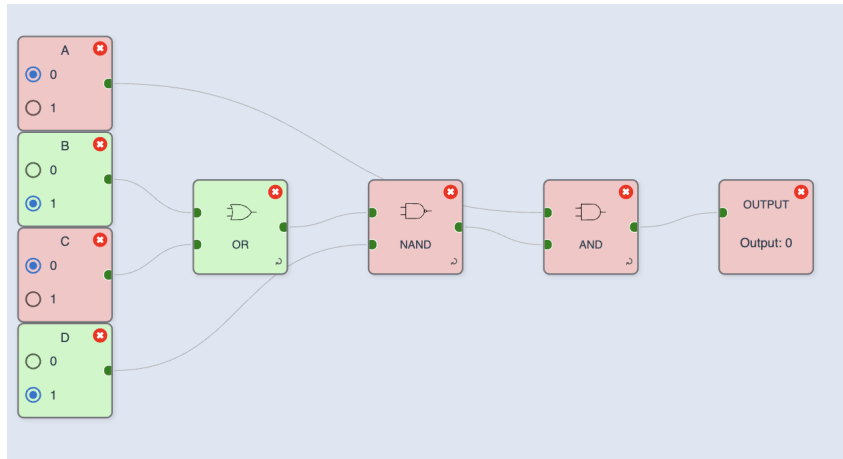


Figure 5.6: Cascading Colour-Coded Nodes as Binary Values Traverse the Circuit

### Node Flipping

Users can click on the "flip" icon on gate nodes to see the intermediate working of each gate given the input combination that it is receiving, along with a truth table on hover, representing the intermediate computational step highlighted in bold. This information is provided in the circuit itself and in a minimal, compact manner.

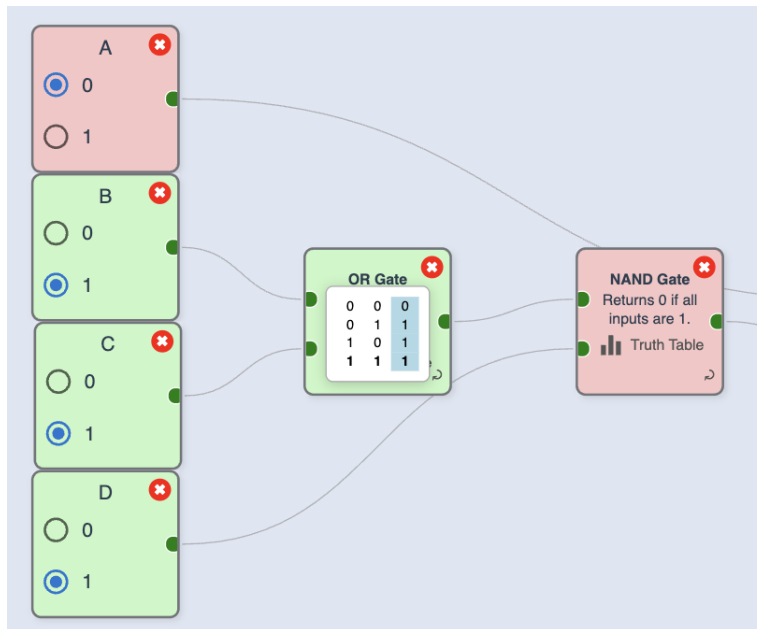


Figure 5.7: Real-time Update of Mini Truth Table for Each Node

## Tooltip Hover Effects

Each node in the toolbar displays a helpful tooltip describing what node the symbol represents. For input nodes, after 10 nodes have been used in the circuit, the button gets disabled in the toolbar and a tooltip appears explaining the limit on the number of input nodes.



Figure 5.8: Tooltips for Each Node in Toolbar

## Resetting Canvas

There is a reset button in the toolbar which clears the canvas and existing computation. Once clicked, it displays a tooltip describing the action taken.

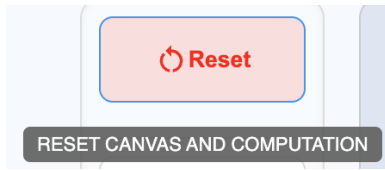


Figure 5.9: \*

(a) Reset Button

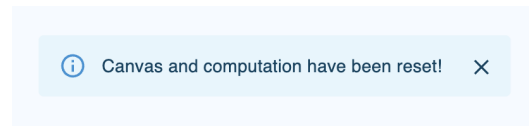


Figure 5.10: \*

(b) Reset Tooltip

Figure 5.11: Reset functionality: button and tooltip

## Terminal Coloring

To ensure sound logic of the nodes, the handles have been modified such that it is only possible to have a one-is-to-one connection between two handles. The left side handles are input terminals and the right side handles are output terminals. When a terminal is connected, it turns green and when the connected node is deleted, its edge gets deleted and it turns red again, indicating that it is available for connection again.

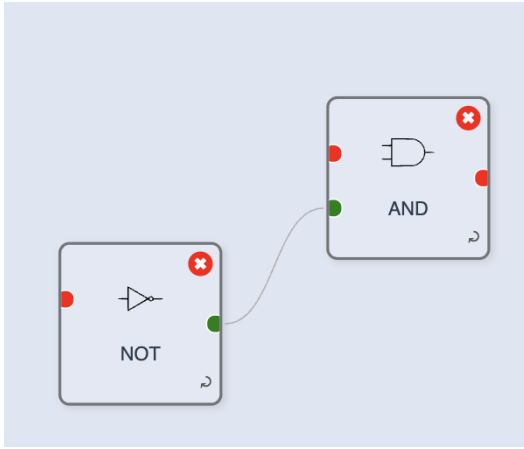


Figure 5.12: \*

(a) Disconnected Terminal (Red)

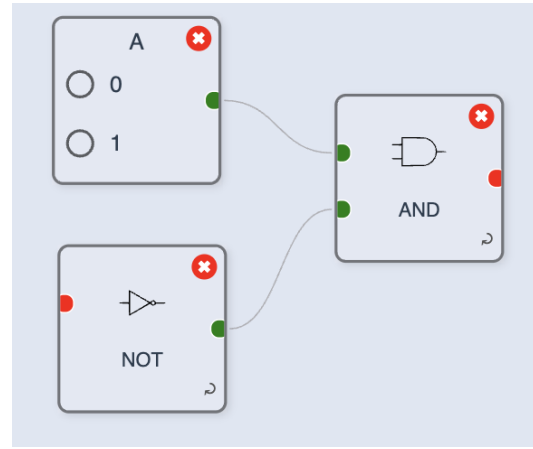


Figure 5.13: \*

(b) Connected Terminal (Green)

Figure 5.14: Enabling and Disabling of Terminals Based on Connection State

### Truth Table Features

Large tables are paginated to avoid information overload. Computation by default computes 10 rows and has an option to "See More Rows" which generates 10 more subsequent rows in the table. The first row in the table is highlighted in blue, and it represents the input combination selected by the user in their circuit diagram. Recomputation highlights the new active row. The number of rows generated in the truth table are mentioned at the top, and the number of truth and falsehood values are also mentioned per column. The user is able to filter the truth table columns singularly and compositely and clear filters. The metadata provided updates whenever recomputation occurs or filters are modified.

Figure 5.15 highlights these GUI elements that provide a subtle way of enhancing user experience by providing useful data.

Generated Truth Table

Rows Generated: 13/16 [CLEAR ALL FILTERS](#)

Number of rows generated with this filter / Total number of input combinations

| A                  | B                  | C                  | D                  | Output              |
|--------------------|--------------------|--------------------|--------------------|---------------------|
| false: 8   true: 5 | false: 7   true: 6 | false: 7   true: 6 | false: 8   true: 5 | false: 0   true: 13 |
| 0                  | 0                  | 1                  | 0                  | 1                   |
| 0                  | 0                  | 0                  | 0                  | 1                   |
| 0                  | 0                  | 0                  | 1                  | 1                   |
| 0                  | 0                  | 1                  | 1                  | 1                   |
| 0                  | 1                  | 0                  | 0                  | 1                   |
| 0                  | 1                  | 0                  | 1                  | 1                   |
| 0                  | 1                  | 1                  | 0                  | 1                   |
| 0                  | 1                  | 1                  | 1                  | 1                   |
| 1                  | 0                  | 0                  | 0                  | 1                   |
| 1                  | 0                  | 0                  | 1                  | 1                   |

SEE MORE ROWS

Pagination- 10 rows visible at a time

Figure 5.15: Truth Table Features

## Step-by-Step Reduction

In propositional mode, each transformation is broken down step-by-step with visual clarity. The formula of the circuit is provided, along with switchable tabs that allow the user to view the reduction in conjunctive and disjunctive normal forms. Each step is explained in detail- taking subphrases from the formula and replacing them with the expanded and simplified versions.

## React Tour Integration

A welcome modal appears on load which briefly summarises the functionality of the app and provides the option to tour the project. The guided tutorial introduces users to core features of the application and what the visual elements do. The React-Tour library was used for this feature. It is shown on first visit and then again when the page is loaded (i.e. it is stored in localStorage).

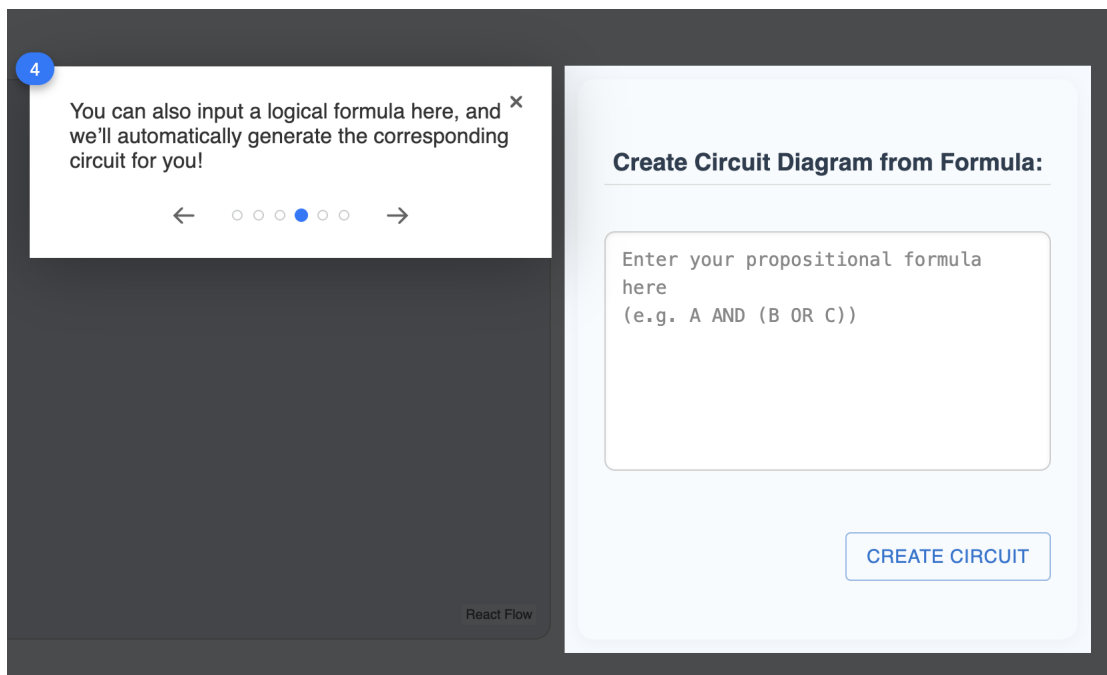


Figure 5.16: Tutorial takes User Through Each Feature

### 5.5.5 Reliability, Feedback and Error Prevention

The application provides thorough checks and feedback to users through validations and error management, aiming to guide rather than confuse:

- Invalid formulas return precise error strings displayed below the input box.
- Circuit restrictions on the number of terminals being inputted and outputted disallow multiple terminals from connecting to a single point and preventing circular loops.
- The **Compute** button checks for errors before triggering backend computation [See Figure 5.17]. It responds with specific errors that inform the user exactly what changes need to be made to make their circuit valid.

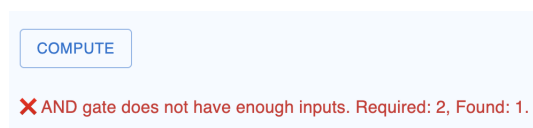


Figure 5.17: Example of an Error for an Invalid Circuit

## Chapter 6

# Implementation

### 6.1 Brief

This chapter is a walkthrough for the technical execution of the program. Each section delves into the frontend and backend core handling for various key features such as circuit visualisation, truth table generation and propositional reduction. This section expands into the implementation of our structure as described in the design chapter.

### 6.2 Code Structure

The code has been structured into two main directories, namely `backend` and `frontend`. The `backend` directory runs a Flask application that handles logic parsing, truth table generation, and formula transformations. The `frontend` directory runs a React app (`npx create-react-app`) and is responsible for handling the canvas creation, the error display and the UI structure. It accesses the data in the backend with calls to the APIs defined in `app.py`.

#### 6.2.1 Frontend Structure

The frontend renders the App component in `Index.js`, and the routing is handled in `App.js`. Initially, there were multiple pages in the application and therefore `react-router-dom` was used to configure routes and navigation. However, now, there is a single page and therefore a single route, which is the base route for the app. `App.css` contains the central styling for the app, which helps it maintain a consistent theme.

The main structure inside `src/` is as follows:

- **components/**: Reusable UI components such as the node toolbar, canvas area, and buttons.
- **icons/**: Gate icons as svg files.
- **pages/**: Folder to store all the routes (Currently contains only one file but this folder organisation is useful if other pages want to be added as a part of the future vision). This folder contains the file `UseTool.js`, which is our main file in which all components are rendered, canvas is created, and tab switching is handled.
- **subtabs/**: Contains the tab-specific components present under the compute section of the page: `TruthTableGeneration.js` and `PropositionalReduction.js`
- **utils/**: Utility functions are contained in this file. It contains `validateCircuit.js` which is a reusable file that validates the circuit in the canvas and returns specific error strings.

### 6.2.2 Backend Structure

The backend has its entry point in a file called `app.py`, where all the APIs are configured. The APIs, on getting called, further call the functions written in `backend/logic`. This folder contains the main implementation of the logic used in the app. It has the following main files:

- **formula-to-circuit.py**: Parses the formula string, returns errors if invalid else returns circuit data in the form of nodes and connections.
- **propositional-reduction.py**: Receives circuit data and creates an Abstract Syntax Tree; returns a JSON file containing the propositional formula of that circuit, the reduced formulas in normal forms and the steps of the reduction.
- **truth-table.py**: Receives circuit data and extracts relevant information about the input nodes and evaluates the output for every possible combination of input nodes given the structure of the circuit by building a graph from the circuit information and introducing a topological sorting algorithm.

## 6.3 Canvas Creation and Dynamic Visualisation

The central focus of the tool is an interactive canvas functionality. The drag-and-drop nodes, connections, value propagation, color-coding, and dynamic updation were made possible through

React's library (**React-Flow**) and robust state management and side effect handling. Approaches to leverage this have been outlined hereafter.

### 6.3.1 Node Component Hierarchy: Prop Handling and Node States

The nodes and colour-coding in the canvas involves using a React component hierarchy where each node on the canvas has a **Node** wrapper component which is where the rendering of all the types of nodes takes place. This allows reusability of code, compactness and seamless styling. The state handling is kept global (central) and props are passed down the hierarchy till they are eventually used and modified, which trigger side effects (**UseEffect()** hooks).

In **UseTool.js**, custom node types are registered via a **nodeTypes** object. The toolbar selection sets the selected node type.

Listing 6.1: Custom Node Types

```
const nodeTypes = {  
  INPUT: Node, OUTPUT: Node, AND: Node, OR: Node, NOT: Node, NAND: Node, NOR: Node, XOR  
    : Node, XNOR: Node,};
```

A terminal configuration is also provided for each type of node, indicating the number of inputs and outputs each node will take.

Listing 6.2: Terminal Configuration by Node Type

```
const terminalConfig = {  
  INPUT: { inputs: 0, outputs: 1 },  
  OUTPUT: { inputs: 1, outputs: 0 },  
  NOT: { inputs: 1, outputs: 1 },  
  AND: { inputs: 2, outputs: 1 },  
  // ... other gates  
};
```

### Global State

The node data is stored in a global state in the form of the elements themselves, the values of the nodes, the input values they are receiving and the node type. These shared props are passed down to each node to keep the logic synchronized.

Each time a node is created on the canvas, it is given data such as its terminal count, label, and callback references.

Listing 6.3: Adding New Node On Pane Click

```
const newNode = {
  id,
  type: selectedNodeType,
  data: {
    id,
    label,
    onDelete: handleDelete,
    inputs: terminalConfig[selectedNodeType]?.inputs || 0,
    outputs: terminalConfig[selectedNodeType]?.outputs || 0,
    updateNodeValue,
    inputValues: inputValues[id] || [],
  },
  position,
};
```

## Rendering Specific Node Components

When adding a new node, the custom node rendering is funneled through a single `Node` component, which in turn further passes down the props to other, more specific node components: `GateNode`, `InputNode`, or `OutputNode`:

Listing 6.4: Dynamic Rendering of Specific Node

```
switch (type) {
  case 'INPUT':
    return <InputNode id={id} label={label} updateNodeValue={updateNodeValue} />;
  case 'OUTPUT':
    return <OutputNode id={id} inputValues={inputValues} updateNodeValue={
      updateNodeValue} />;
  default:
    return <GateNode id={id} gateType={type} inputValues={inputValues}
      updateNodeValue={updateNodeValue} />;
}
```

This structure allows a consistent data flow- every node is given the same information and interacts the same way and eventually rendered in a specialized manner with unique behaviour and style.

## Dynamic Input Evaluation

Whenever node values or connections change, a `useEffect` hook recalculates the inputs for each node by traversing the current edge list. This ensures logical correctness across the circuit. Subsequently, this triggers another `useEffect`, which changes the input values in the nodes and passes the new data as a prop down the node hierarchy. This allows the colour-coding to happen in real-time.

Listing 6.5: Computing Inputs based on Edges

```
useEffect(() => {
  const newInputValues = {};
  edges.forEach(({ source, target }) => {
    if (!newInputValues[target]) newInputValues[target] = [];
    newInputValues[target].push(nodeValues[source] ?? null);
  });

  elements.forEach((node) => {
    if (node.type === "INPUT") {
      newInputValues[node.id] = [nodeValues[node.id] ?? null];
    }
  });

  setInputValues(newInputValues);
}, [edges, nodeValues]);
```

## Modularity and Reusability

This structure allows each node to behave as a controlled component—taking in values through props and updating the global state via shared callback functions. By managing everything centrally using React hooks like `useEffect`, `useCallback`, and `useRef`, the canvas remains consistent and easier to debug. It also allows for flexibility and scalability: for future introductions of new gate types, or allowing multiple terminals, the corresponding configuration can be updated without changing the structure or the logic.

### 6.3.2 Edges and Node Terminals: Mapping and Dynamic Rendering

To make the edges sound and reliable with the theory of logic circuits, react-flow handles were modified to only take one-to-one edges between handles. Additionally, they were modified to be defined as source and target handles, indicating that source (input) handles of nodes could not connect with each other. More than one input handle was added for the gate nodes as they take two inputs and produce one output (exception: NOT gate).

An `edges` state stores the edges present in the canvas. React-Flow's `onConnect` function was overwritten to update the `edges` state when connections between handles were made.

Changes in the `edges` state trigger a `useEffect()` in which a `ConnectionMap` is initialised and filled, whose objective is to track the node handles that are currently connected or disconnected.

The `ConnectionMap` object refers to the current `elements` state, i.e. the data of the nodes present in the canvas. It maps each node by id to an inputs array and outputs array and sets it to false. Then, depending on the edges changed, it updates the relevant array to true if that node is involved in a connection.

Listing 6.6: Initialising the Connection Map

```
const connectionMap = {};  
  
// Initialize the map with input and output handle arrays based on the node data  
elements.forEach(node => {  
  if (!node.data) return;  
  
  connectionMap[node.id] = {  
    input: Array(node.data.inputs || 0).fill(false),  
    output: Array(node.data.outputs || 0).fill(false),  
  };  
});
```

This map then passes an `isConnected` prop to the Node component. The Node component renders handles for each node and updates them, i.e. enables them when they are disconnected (style: red) and disables them when they are connected (style: green; connections are no longer possible). This ensures one-to-one mapping of the terminals.

Listing 6.7: Rendering input handles based on connection state

```
{Array.from({ length: inputs }).map((_, index) => (  
  <Handle  
    key={`input-${index}`}  
    type="target"  
    position={Position.Left}  
    id={`input-${index}`}  
    className={`handle-input ${isConnected?.input?.[index] ? 'connected' : ''}`}  
    style={{  
      top: `${(index + 1) * 40}px`,  
    }}  
  />  
)})}
```

### 6.3.3 Handler Functions

In addition to node and edge creation, several helper functions were also used in creation of the circuit- during prop handling as well as for React-Flow rendering. They are outlined as follows:

- **Deletion Logic:** The `handleDelete` function is passed down as a prop down the node component hierarchy. When it is called, the node it is called on is removed from the canvas and the `elements` state is updated. Any edges that node was connected to are also deleted.
- **Input Node Labeling and Limits:** React's `useRef` hook is used to manage stable input node IDs. A maximum of ten input nodes is allowed, labeled from A to J. If a label is freed via deletion, their labels (e.g., A–J) are released for reuse, making them available to be assigned to new input nodes.
- **Canvas Reset:** The `handleReset` functionality clears all nodes and edges from the canvas and resets internal state variables. It also sends a signal to the computational elements, namely truth table and propositional reduction, and their states are cleared as well.
- **Node Dragging:** The `onNodeDrag` handler updates a node's position in the internal state when it is moved on the canvas. This is done by modifying the position variable in the data of the node.

## 6.4 Computational Section – Validation

Before any logic computation is carried out or API calls are made, the circuit first goes through a validation phase to ensure it has been constructed in accordance to logic circuit principles. When the user clicks the **Compute** button in either subtab in the computational section, the `validateCircuit()` function is called (present in `util/`). The aim of this is to catch any errors before processing begins to prevent incorrect data or unexpected behaviour.

Internally, the validation function creates a directed graph for the circuit. This representation helps in understanding the flow of logic from input to output. Alongside this, a reverse graph is also built—allowing the system to trace backwards from the output node to make sure all nodes are reachable and there is no isolated or disconnected node. Another reason for the reverse graph is to check for loops, although the UI already disallows that from happening. The function performs several key checks:

- **Only one output node is allowed:** If more than one output node is present, the validation fails. The system is designed to handle a single point of output, and multiple outputs are currently not supported.
- **An output node must exist:** The circuit must have at least one output node defined. If it's missing, the circuit has no end point for evaluation and is considered invalid.
- **All terminals must be connected:** Every input and output handle of each node must be connected to something. The validation compares the number of expected terminals (based on the gate type) with the actual connections made.
- **Inputs must be binary:** Each input node must be toggled to either a 0 or 1 value before computation. If an input hasn't been assigned a value, the validation stops.
- **Correct number of inputs per gate:** Each gate must have the expected number of inputs wired in. For instance, an AND gate needs two inputs, while a NOT gate only needs one. Any mismatch is caught during this stage.

This validation process ensures that users do not accidentally compute on incomplete or incorrectly formed circuits.

## 6.5 Truth Table Generation

### 6.5.1 Frontend: Rendering and Filtering

The **TruthTable** component in the frontend is responsible for rendering the computed output received from the backend via the API call in a structured, user-friendly, interactive format.

The component accepts a **data** prop, which includes an array of input variable names and a truth table (an array of objects representing each input combination and its corresponding output- this has been provided by the backend).

The internal states include:

- **visibleCount**: Controls how many rows are rendered at a time. It is set to 10. This supports a "See More Rows" feature-Pagination- to incrementally load more data.
- **filters**: An object that tracks per-column filters. Users can filter input or output columns to only display rows with specific values (0 or 1).
- **anchorEls**: Used to view the dropdown and close it.

Whenever a filter is applied, the component computes a **filteredTable** by checking that each row satisfies all active filter conditions. From this filtered dataset, only the first 10 rows are rendered initially.

Each input and output column includes:

- A label showing the variable name.
- A filter icon that toggles a dropdown, allowing user to filter by 0 or 1 or remove a filter..
- A count summary showing how many rows in the current view have true (1) or false (0) values for that column.

The output column is visually separated using a distinct background color. Additionally, the backend marks "highlighted":True for the row containing the current input combination. Therefore, the current input combination row and output is present at the top of the table and is highlighted.

The structure allows for efficient user interaction even with large datasets and helps isolate specific logical patterns through filtering. The entire component is styled using MUI's **Box**, **Typography**, and **Table** elements to ensure consistency.

## 6.5.2 Backend: Computation and Processing

On the backend, the logic for truth table generation is handled by the `generate_truth_table` function, which processes the user-defined circuit and computes output values for all possible combinations of input values.

### Parsing Circuit Data

The function parses the input `circuit_data` dict, which contains two primary elements- the nodes and the connections.

- `nodes`
- `connections` – edges between nodes, each with a source and target ID.

Input nodes are filtered from the node list by type, and their IDs are used to generate all possible combinations of input values. These IDs are also used as input names in the final output.

### Graph Construction

Each node in the circuit is converted into a dictionary containing its type, list of input and output node IDs, and a temporary value. The `graph` dictionary stores these nodes. Then, the graph iterated over the `connections` and populates the `inputs` and `outputs` data for each nodes:

- For each connection, the target node is appended to the source node's `outputs` list.
- The source node is appended to the target node's `inputs` list.

Thus, the circuit successfully gets converted into a directed graph, which the backend now uses to evaluate the output for different input combinations.

### Evaluating the Circuit

The evaluation is performed for all input combinations according to the pseudocode described in the algorithm given below.

If an error occurs during evaluation (such as a node having undefined input), the output is marked as "?" to indicate that evaluation failed for that row. Otherwise, the combination and its resulting output are added to the final truth table.

---

**Algorithm 1** Topological Traversal and Evaluation of Logic Circuit

---

```
1: Input: input_combination, circuit_nodes, circuit_connections
2: Output: output_node_value
3: input_values  $\leftarrow$  assign_values_to_inputs(input_combination)
4: graph  $\leftarrow$  build_graph(circuit_nodes, circuit_connections)
5: queue  $\leftarrow$  initialize_queue(graph)
6: while queue is not empty do
7:   current_node  $\leftarrow$  queue.dequeue()
8:   if current_node is a logic gate then
9:     input_values_for_gate  $\leftarrow$  get_input_values(current_node, graph)
10:    if all inputs are ready (no None values in input_values_for_gate) then
11:      output_value  $\leftarrow$  compute_gate_output(current_node.type,
        input_values_for_gate)
12:      current_node.value  $\leftarrow$  output_value
13:      for each output_node in current_node.outputs do
14:        if output_node is not in queue then
15:          queue.enqueue(output_node)
16:        end if
17:      end for
18:    end if
19:  end if
20: end while
21: output_node_value  $\leftarrow$  get_output_value_from_graph(graph)
22: return output_node_value
```

---

## 6.6 Propositional Reduction Tool

### 6.6.1 Frontend

The frontend of the Propositional Reduction Tool is responsible for displaying the step-by-step transformation of a propositional formula into its equivalent CNF (Conjunctive Normal Form) and DNF (Disjunctive Normal Form) representations.

- The interface is divided into two subtabs: **CNF** and **DNF**, each rendering a similar layout to maintain a consistent user experience.
- Upon computing a formula, both CNF and DNF transformations are received as ordered lists of transformation steps from the backend. Each step includes a "**before**" and "**after**" string representing the transformation.
- The frontend iteratively renders these steps using a numbered list. For each step, it highlights the part of the formula that was changed and displays the updated formula after the transformation.
- Internally, the frontend maintains local component state to track and persist computed results, ensuring that switching between tabs does not cause data loss.

- If the user modifies the input formula or the circuit structure (from which the formula was originally derived), the state is automatically cleared and refreshed to reflect the updated data.
- All formula displays are styled using monospace fonts for better readability and to reflect the logical structure.

### 6.6.2 Backend

The Propositional Reduction backend implements a sophisticated Abstract Syntax Tree (AST) approach with robust transformation algorithms for converting logical formulas to normal forms. The system’s innovation lies in its dual-tree context-preservation mechanism, which enables detailed step-by-step explanations of transformations.

#### Tree-Based Formula Representation

At the core of the implementation is a comprehensive class hierarchy for representing logical formulas:

Listing 6.8: Formula Class Hierarchy

```
class Formula: # Abstract base class
    def __str__(self): ... # String representation
    def negate(self): ... # Logical negation
    def simplify(self): ... # Simplification rules

class Literal(Formula): ... # Variables (A, B, etc.)
class Not(Formula): ... # Negation (A)
class And(Formula): ... # Conjunction (A B)
class Or(Formula): ... # Disjunction (A B)
class True_(Formula): ... # Logical constant TRUE
class False_(Formula): ... # Logical constant FALSE
```

Each formula type implements specialized methods for simplification, negation, and string representation. This object-oriented approach enables processing of formula nodes during reductions.

## Circuit-to-AST Transformation

The system constructs an AST from circuit data using recursive traversal from output to input nodes. The pseudocode for constructing the AST is provided below.

---

**Algorithm 2** Circuit to AST Construction

---

```
1: function BUILDFORMULA(circuit, nodeId)
2:   if nodeId is null then
3:     nodeId  $\leftarrow$  find_output_node(circuit)
4:   end if
5:   node  $\leftarrow$  circuit.nodes[nodeId]
6:   if node.type is "INPUT" then
7:     return new Literal(nodeId)
8:   end if
9:   childIds  $\leftarrow$  [conn.source for conn in circuit.connections where conn.target = nodeId]
10:  childFormulas  $\leftarrow$  [BuildFormula(circuit, childId) for childId in childIds]
11:  if node.type is "AND" then
12:    return new And(childFormulas)
13:  else if node.type is "OR" then
14:    return new Or(childFormulas)
15:  else if node.type is "NOT" then
16:    return new Not(childFormulas[0])
17:  else if node.type is "XOR" then
18:    return buildXorEquivalent(childFormulas)
19:  else
20:    return appropriate_formula_for_gate_type(node.type, childFormulas)
21:  end if
22: end function
```

---

Complex gates (XOR, NAND, etc.) are decomposed into their equivalent expressions using basic operations. For example, XOR is represented as  $(A \wedge \neg B) \vee (\neg A \wedge B)$ .

## Dual-Tree Context Preservation Architecture

The most innovative aspect of our implementation is the dual-tree architecture that maintains context awareness during recursive transformations. This system solves a fundamental challenge in formula transformation algorithms: preserving the complete formula context while transforming individual subexpressions.

Listing 6.9: Context Preservation Mechanism

```
def rebuild_formula(current, other_branches, junctions):
    """Reconstruct complete formula from current branch and context history"""
    if not other_branches or not junctions:
        return current

    # Work backwards through branch history
    rebuilt = current
    for branch, junction_type in zip(reversed(other_branches), reversed(junctions)):
        if junction_type == 'AND':
            rebuilt = And(branch, rebuilt)
        elif junction_type == 'OR':
            rebuilt = Or(branch, rebuilt)

    return rebuilt
```

During recursive traversal, the system maintains two parallel contexts:

1. The **current transformation point** — the subformula being actively transformed
2. The **contextual path** — a record of how to reconstruct the complete formula

This approach functions similarly to a stack trace in programming languages, letting us "rewind" from any depth of recursion to rebuild the complete formula context.

### De Morgan's Law Application with Context Tracking

The application of De Morgan's laws pushes negations inward, applying the following transformations:

- $\neg(A \wedge B) \rightarrow (\neg A \vee \neg B)$
- $\neg(A \vee B) \rightarrow (\neg A \wedge \neg B)$
- $\neg\neg A \rightarrow A$

Our implementation handles this recursively while maintaining branch tracking:

---

**Algorithm 3** De Morgan's Application with Context Tracking

---

```

1: function APPLYDEMORGAN(formula, steps, step_num, other_branches, junctions)
2:   if formula is atomic (Literal, TRUE, FALSE) then
3:     return formula, steps, step_num, other_branches, junctions
4:   end if
5:   if formula is Not then
6:     inner  $\leftarrow$  formula.formula
7:     if inner is And then  $\triangleright$  Apply  $\neg(A \wedge B) \rightarrow (\neg A \vee \neg B)$ 
8:       result  $\leftarrow$  Or(*[Not(f) for f in inner.formulas])
9:       full_formula  $\leftarrow$  RebuildFormula(result, other_branches, junctions)
10:      steps.append({step_num, description, full_formula})
11:      if inner has multiple subformulas then
12:        left_branch  $\leftarrow$  result.formulas[0]
13:        right_branch  $\leftarrow$  result.formulas[1]
 $\triangleright$  Process left branch with context
14:        left_other  $\leftarrow$  other_branches.copy().append(right_branch)
15:        left_junctions  $\leftarrow$  junctions.copy().append('OR')
16:        left_result, steps, left_step  $\leftarrow$  ApplyDeMorgan(left_branch, steps,
step_num+1, left_other, left_junctions)
 $\triangleright$  Process right branch with context
17:        right_other  $\leftarrow$  other_branches.copy().append(left_result)
18:        right_junctions  $\leftarrow$  junctions.copy().append('OR')
19:        right_result, steps, right_step  $\leftarrow$  ApplyDeMorgan(right_branch, steps,
left_step, right_other, right_junctions)
20:        final_result  $\leftarrow$  combine_results(left_result, right_result)
21:        return final_result, steps, right_step, other_branches, junctions
22:      end if
23:    else if inner is Or then  $\triangleright$  Similar logic for  $\neg(A \vee B) \rightarrow (\neg A \wedge \neg B)$ 
24:      ...
25:    end if
26:  end if
 $\triangleright$  Process AND/OR nodes similarly...
27:  ...
28:  return formula, steps, step_num, other_branches, junctions
29: end function

```

---

When a transformation occurs, the system:

1. Transforms the targeted subformula
2. Uses `rebuild_formula` with context data to reconstruct the complete formula
3. Records the step with before/after states and the complete formula

## CNF and DNF Transformation Pipeline

The normal form conversion follows a strategic pipeline:

Listing 6.10: Normal Form Conversion Pipeline

```
def to_cnf(formula):  
    """Convert formula to CNF with step tracking"""  
    steps = []  
    step_number = 1  
  
    # Phase 1: Apply De Morgan's laws to push negations inward  
    formula, steps, step_number, branches, junctions = apply_de_morgans_step(  
        formula, steps, step_number)  
  
    # Phase 2: Iteratively distribute OR over AND until in CNF  
    while not is_in_cnf(formula):  
        # Distribution step  
        formula, steps, step_number, branches, junctions = distribute_or_over_and_step(  
            formula, steps, step_number, branches, junctions)  
  
        # Simplification after distribution  
        formula, steps, step_number, branches, junctions = simplify_step(  
            formula, steps, step_number, branches, junctions)  
  
        # Prevent infinite loops  
        if no_changes_in_last_iteration:  
            break  
  
    return str(formula), steps
```

For DNF conversion, the process is similar but uses `distribute_and_over_or_step` instead. Each phase generates detailed steps that explain the transformation being applied, providing educational value to users.

The `is_in_cnf` and `is_in_dnf` functions perform structural validation to ensure the formula meets the formal definition of the respective normal form. This ensures the algorithm terminates when the desired structure has been achieved.

## 6.7 Formula-to-Circuit Converter

### 6.7.1 Frontend

#### Main Function for API Call: `handleFormulaSubmit()`

This is the function that handles the submission of the input string and calls the API. If the API returns an error string, it is formatted and returned to the user. Otherwise, the data received is stored in a temporary node array and connections array. IDs are assigned at the frontend level to these nodes and the connections are modified to be compatible with these node IDs. Following this, the states for `elements` and `edges` are updated.

#### Helper Function: `prettifyLayout()`

To improve the readability and visual appeal of the generated circuit graph, the frontend writes a function called `prettifyLayout()`. The purpose of this function is to create the circuit by arranging nodes in tree structure. The logical layout is decided on the basis of their connections in a left-to-right format.

The logic of the `prettifyLayout()` function can be broken down as follows:

#### 1. In-degree Computation:

- All the connections are iterated and the in-degree of each target node is incremented.

#### 2. Topological Level Assignment:

- Input nodes are added to the queue and assigned to `level 0`.
- A breadth-first traversal is performed, assigning each node a `level` based on its distance from the input layer.
- For each visited node, all of its children are assigned a level one greater than the current node.

#### 3. Positioning:

- Nodes are grouped into horizontal layers based on their computed level, representing their depth in the graph.
- For each layer,  $(x)$  and  $(y)$  spacing is applied.

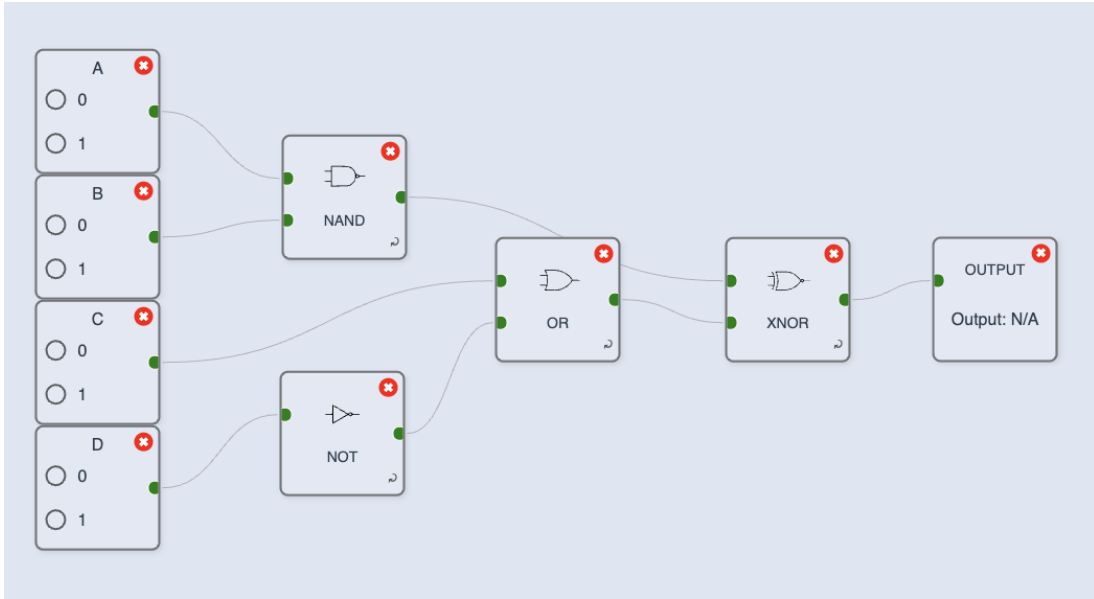


Figure 6.1: Automatically Arranged Nodes using `prettifyLayout()`

This layout ensures a clear left-to-right data flow, visually conveying logical dependencies between gates and reducing overlaps or unnecessary edge crossings.

The function returns a list of nodes with modified position attribute which are used to render the circuit in a sophisticated manner.

### 6.7.2 Backend

The Formula-to-Circuit converter backend implements a sophisticated recursive descent parser that transforms propositional logic expressions into circuit representations. The implementation handles complex grammar validation, operator precedence, and structural correctness while providing informative error messages.

#### Parser Design and Architecture

The parser's architecture centers around the `FormulaParser` class, which encapsulates both the parsing logic and circuit construction.

## Grammar Validation and Error Handling

Before parsing begins, the system performs rigorous validation to catch syntax errors and provide clear error messages to the user. This validation occurs in multiple phases:

---

**Algorithm 4** Formula Grammar Validation

---

```
1: function VALIDATEGRAMMAR(formula)
2:   formula  $\leftarrow$  NormalizeSymbols(formula) ▷ Convert , ,  $\neg$  to text
▷ Check for invalid operator sequences
3:   if formula contains "NOT AND" or "NOT OR" then
4:     return Error("NOT cannot be directly followed by operators")
5:   end if
6:   if formula contains consecutive operators (e.g., "AND OR") then
7:     return Error("Consecutive operators not allowed")
8:   end if
9:   if formula starts with "AND" or "OR" then
10:    return Error("Formula cannot start with binary operators")
11:  end if
12:  if formula ends with any operator then
13:    return Error("Formula cannot end with operators")
14:  end if ▷ Check parenthesis balance
15:  balance  $\leftarrow$  0
16:  for each character c in formula do
17:    if c is '(' then
18:      balance  $\leftarrow$  balance + 1
19:    else if c is ')' then
20:      balance  $\leftarrow$  balance - 1
21:      if balance < 0 then
22:        return Error("Unbalanced parentheses")
23:      end if
24:    end if
25:  end for
26:  if balance  $\neq$  0 then
27:    return Error("Unbalanced parentheses")
28:  end if ▷ Validate literal names
29:  for each literal name in formula do
30:    if name not in A-J then
31:      return Error(f"Invalid literal: name. Only A-J allowed")
32:    end if
33:  end for
34:  return Valid
35: end function
```

---

This multi-stage validation approach helps catch common user errors before circuit generation begins, improving the user experience by providing immediate feedback.

## Recursive Parsing and Operator Precedence

The core parsing algorithm uses recursive descent with operator precedence to build a structured representation of the formula.

A key challenge in parsing logical expressions is handling operator precedence correctly. Our implementation uses a sophisticated algorithm to identify the main operator in an expression while respecting parentheses and precedence rules.

---

**Algorithm 5** Main Operator Identification

---

```
1: function FINDMAINOPERATOR(expr)
2:   precedence  $\leftarrow$  {"AND": 1, "OR": 2, "XOR": 2, "NAND": 1, "NOR": 1, "XNOR": 1}
3:   paren_level  $\leftarrow$  0
4:   operators  $\leftarrow$  []
5:   for i  $\leftarrow$  0 to length(expr)-1 do
6:     if expr[i] is '(' then
7:       paren_level  $\leftarrow$  paren_level + 1
8:     else if expr[i] is ')' then
9:       paren_level  $\leftarrow$  paren_level - 1
10:    else if paren_level = 0 then
11:      for each operator_type in supported_operators do
12:        if expr[i:i+len(operator_type)] matches operator_type then
13:          if operator is a complete word (surrounded by spaces/parens) then
14:            operators.append((operator_type, i, precedence[operator_type]))
15:          end if
16:        end if
17:      end for
18:    end if
19:  end for
20:  if operators is empty then
21:    return null
22:  end if
   $\triangleright$  Find operator with lowest precedence (highest value)  $\triangleright$  If tied, choose rightmost one
  (for right-associativity)
23:  main_op  $\leftarrow$  max(operators, key=lambda x: (x[2], x[1]))
24:  op  $\leftarrow$  main_op[0]
25:  pos  $\leftarrow$  main_op[1]
26:  left_expr  $\leftarrow$  expr[:pos]
27:  right_expr  $\leftarrow$  expr[pos+len(op):]
28:  return (op, left_expr, right_expr)
29: end function
```

---

This algorithm ensures that expressions like "A AND B OR C" are correctly interpreted as "(A AND B) OR C" based on operator precedence, without requiring explicit parentheses.

The combination of these components creates a robust parser that converts logical formulas into properly structured circuit diagrams, enhancing the user experience through clear error messages and reliable conversion.

## Chapter 7

# Testing and Evaluation

### 7.1 Brief

This chapter presents the measures taken to ensure the correctness, reliability, and performance of the LogiDex system. It details the automated unit tests implemented during backend development, the user testing carried out with a small cohort, and a critical evaluation of the tool against its non-functional requirements.

### 7.2 Automated Unit Testing

To ensure robustness and correctness of the backend logic, comprehensive unit tests were written and organized under the `backend/tests/` directory. The testing framework used is `pytest`, with configurations set in the `pytest.ini` file. A central script, `run-tests.py`, is used to initialize and run the test suite efficiently.

The test files are categorized as follows:

- **`test_formula_to_circuit.py`**: Verifies that propositional formulas are correctly parsed and converted into valid circuit representations.
- **`test_logic_integration.py`**: Performs integration testing of full workflows, such as end-to-end formula generation, truth table evaluation, and state updates.
- **`test_propositional_reduction.py`**: Checks the accuracy of CNF and DNF reductions, including step-by-step rule application.

- **test\_truth\_table.py:** Confirms that truth tables are generated correctly, including edge cases and circuits with various depths and branching.

These tests cover both unit-level logic and integration across modules, helping prevent regressions and ensuring future extensibility.

## 7.3 User Testing and Manual Testing

User testing was conducted, specifically for the frontend, to evaluate the usability, functionality, and general user experience of LogiDex, since it is a dynamic app with lot of changes on manipulation.

**User testing** refers to a process where real users interact with a system to identify issues and areas of improvement. It allows developers to observe how the tool performs in practical use and ensures alignment with user expectations. As Nielsen (1994) notes, usability testing is an irreplaceable practice in user-centered interaction design, as it provides direct input on how real users interact with the system [11].

An iterative design principle was chosen. A cohort of 10 participants used the software and interacted with all key features, including circuit creation, formula generation, truth table output, and propositional logic reduction. They were encouraged to report bugs and usability issues, many of which were incorporated into the final refinement process. This helped improve UI consistency, error handling, and accessibility features.

## 7.4 Evaluation

This section looks at how well LogiDex met the non-functional requirements (NFRs) mentioned earlier. Each requirement was tested through browser checks, performance observation, or feedback from actual users.

### NFR-01: Performance

**Upon evaluation:** Most interactions such as toggling inputs or generating outputs responded within 150 to 200ms across all major browsers.

**Finding:** The system stays quick and responsive.

**Conclusion:** NFR-01 satisfied.

#### **NFR-02: Scalability**

**Upon evaluation:** Circuits with up to 50 nodes were tested and ran without noticeable lag. Outputs were computed in under a second.

**Finding:** The system handles large circuits smoothly.

**Conclusion: NFR-02 satisfied.**

#### **NFR-03: Usability**

**Upon evaluation:** Error messages were clear and specific, often suggesting exactly what needed fixing.

**Finding:** Error handling is intuitive and helpful.

**Conclusion: NFR-03 satisfied.**

#### **NFR-04: Design Consistency**

**Upon evaluation:** The layout remained neat and readable even after dragging nodes or zooming in and out.

**Finding:** The visual structure is consistent and reliable.

**Conclusion: NFR-04 satisfied.**

#### **NFR-05: Accessibility**

**Upon evaluation:** Key features could be accessed using keyboard navigation and color contrast met accessibility standards.

**Finding:** The interface is accessible and inclusive.

**Conclusion: NFR-05 satisfied.**

#### **NFR-06: Responsiveness**

**Upon evaluation:** The app scaled correctly on mobile, tablet, and desktop. No layout breaks were found during resizing.

**Finding:** The design adjusts well to screen size.

**Conclusion: NFR-06 satisfied.**

#### **NFR-07: Memory Session**

**Upon evaluation:** Data such as truth tables and reduced formulas stayed intact when switching tabs unless the circuit was changed.

**Finding:** Temporary session state works as intended.

**Conclusion:** NFR-07 satisfied.

#### **NFR-08: Maintainability**

**Upon evaluation:** Code is modular and well-tested, with each component separated cleanly. Changes are easy to make and test.

**Finding:** The project is structured for long-term maintenance.

**Conclusion:** NFR-08 satisfied.

#### **NFR-09: Reliability**

**Upon evaluation:** Circular logic was detected and blocked. The system avoided infinite loops or crashes in all test cases.

**Finding:** Logical safeguards work reliably.

**Conclusion:** NFR-09 satisfied.

#### **NFR-10: Startup Performance**

**Upon evaluation:** The main interface and tutorial loaded in under a second during testing on common browsers.

**Finding:** LogiDex starts up quickly and smoothly.

**Conclusion:** NFR-10 satisfied.

## Chapter 8

# Legal, Social, Ethical and Professional Issues

Care was taken during the development to ensure that illegal, social, professional, and ethical standards were upheld. All third-party libraries and visual assets (such as icons or fonts) used in the project have been clearly attributed, both in the source code and in this report. Academic sources that influenced the design or implementation of key components have been referenced.

The British Computer Society (BCS) Code of Conduct [14] was used as a key ethical framework during development. Efforts were made to ensure the boundaries of personal technical competence, to avoid causing harm, and to act with integrity and transparency at every stage. The platform was built as an educational tool, and at no point was it intended to mislead, misinform, exploit, or manipulate users.

User privacy and data sensitivity were also considered. All session data is stored temporarily in the user's local browser storage, and no server-side logging of user input occurs. These practices align with the principles of the General Data Protection Regulation (GDPR) [1], which emphasizes data minimization and user consent.

During the testing phase, a group of 10 users participated voluntarily, with no material incentive provided. This helped ensure that feedback was unbiased and genuinely reflective of the tool's usability and design. The participants were aware of the project's academic context and gave verbal consent to provide feedback without any personal data being recorded.

Professionally, the software follows modern development practices such as modular code structure, testing, and documentation. Best practices from software engineering ethics, including those from the ACM Code of Ethics [7], were considered, especially regarding software reliability, fairness, and respect for user autonomy.

Overall, the project has been built with care and attention to legal, ethical, and professional responsibilities, ensuring that it is both technically sound and socially responsible.

## Chapter 9

# Conclusion and Future Work

### 9.1 Conclusions

This project demonstrates that visualising logical circuits in an interactive, educational context is both technically feasible and effective. One of the most significant findings was the suitability of React for building sophisticated browser-based tools that function on live feedback and dynamic manipulation. Its component-based structure and local storage allowed for scalable development of interactive modules such as the canvas, truth table viewer, and formula panels, all while maintaining responsiveness and state management even in large circuits.

Another key takeaway was the viability of symbolic-to-visual transformations. The system shows that propositional logic, typically confined to abstract syntax, can be meaningfully rendered into circuits with real-time computation and back-and-forth interactivity. This opens the door for logic-based education tools that move beyond static truth tables and towards hands-on exploration.

Finally, the combination of backend processing for logic evaluation with a responsive frontend interface proved essential. Offloading computation-heavy tasks (like formula reduction or large truth tables) to the backend, while using React on the frontend for real-time updates, enabled the system to perform well, even with more complex inputs. This architecture offers a strong foundation for further expansion into more advanced logic or collaborative educational features.

## 9.2 Future Work

LogiDex currently provides a good range of features for propositional logic exploration with interchangeability between modes of representation, several enhancements are planned to expand its capabilities. The codebase has also been structured in a modular, consistent way to allow for easy expansion of functionality without code duplication or logic interference.

A possible addition is enabling users to define **custom nodes**. This would allow advanced users or educators to create bespoke logic components beyond standard gates, and would even allow compounding preexisting logic gates into a single reusable behaviour.

Another potential feature is adding **support for multiple output nodes** within the same circuit. This would make it possible to model parallel flows in a single workspace. Similarly, support for **support for AND Gates with multiple inputs** would allow more compact and realistic circuit modeling.

The program could also have a **theming system for the UI**, allowing users to switch between different color schemes (e.g., dark mode, high contrast mode) to suit their preferences and be consistent with their system's UI.

In terms of pedagogy, future updates will include dedicated learning pages and functionality, where users can read through a concept and type intermediate logic steps into text boxes, with real-time verification and feedback. This would make the learning experience more active and self-guided.

Each of these features builds naturally on the existing system, and can be easily integrated into the preexisting structure.

# References

- [1] What is gdpr, the eu's new data protection law?, 2024. <https://gdpr.eu/>.
- [2] Academo. Logic gate simulator - academo.org, 2020. <https://academo.org/demos/logic-gate-simulator/>.
- [3] Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh. Handbook of satisfiability, 2009.
- [4] CircuitVerse. Circuitverse - online digital logic circuit simulator, 2020. <https://circuitverse.org>.
- [5] Wikipedia contributors. Boolean algebra, 2024. [https://en.wikipedia.org/wiki/Boolean\\_algebra](https://en.wikipedia.org/wiki/Boolean_algebra).
- [6] Edsger W. Dijkstra. Shunting yard algorithm. In *EWD Manuscripts*, 1961.
- [7] Association for Computing Machinery. Acm code of ethics and professional conduct, 2018. <https://www.acm.org/code-of-ethics>.
- [8] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, 2nd edition, 2004.
- [9] Joel Jones. Abstract syntax tree implementation idioms, 2024.
- [10] Logicly. Logicly - logic circuit simulator, 2020. <https://logic.ly>.
- [11] Jakob Nielsen. *Usability Engineering*. Academic Press Inc, Boston, 1994.
- [12] ScienceDemos.org.uk. Logic gates – interactive science simulations, 2020. <https://www.sciencedemos.org.uk/logic-gates.html>.
- [13] Simulator.io. Logic gate simulator, 2020. <https://www.simulator.io>.

- [14] British Computer Society. Bcs code of conduct, 2015. <https://www.bcs.org/membership/become-a-member/bcs-code-of-conduct/>.
- [15] Stanford Encyclopedia of Philosophy. Propositional logic, 2024.
- [16] Ronald J. Tocci, Neal S. Widmer, and Gregory L. Moss. *Digital Systems: Principles and Applications*. Prentice Hall, 12th edition, 2010.
- [17] University of Notre Dame. Compiler construction notes, 2023.

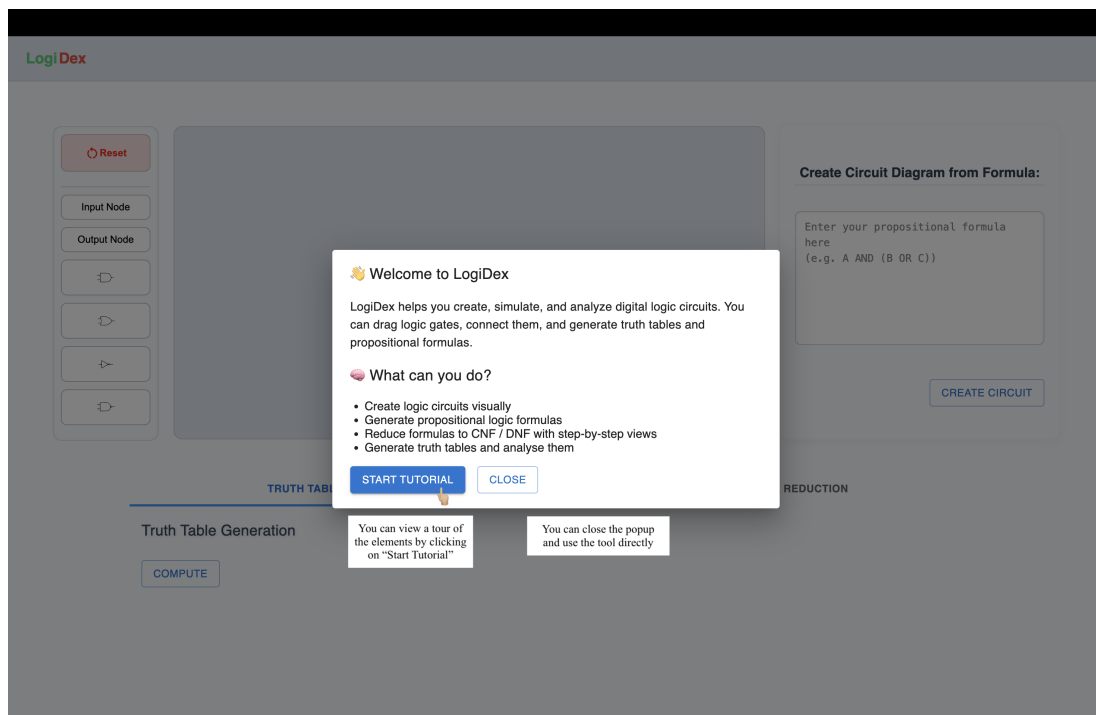
# Appendix A

## User Guide

This user guide provides a step-by-step walkthrough of LogiDex using annotated interface screenshots to illustrate the interactive elements and guide new users through a typical session flow. Each figure contains arrows, boxes, and cues to highlight key features and interactions.

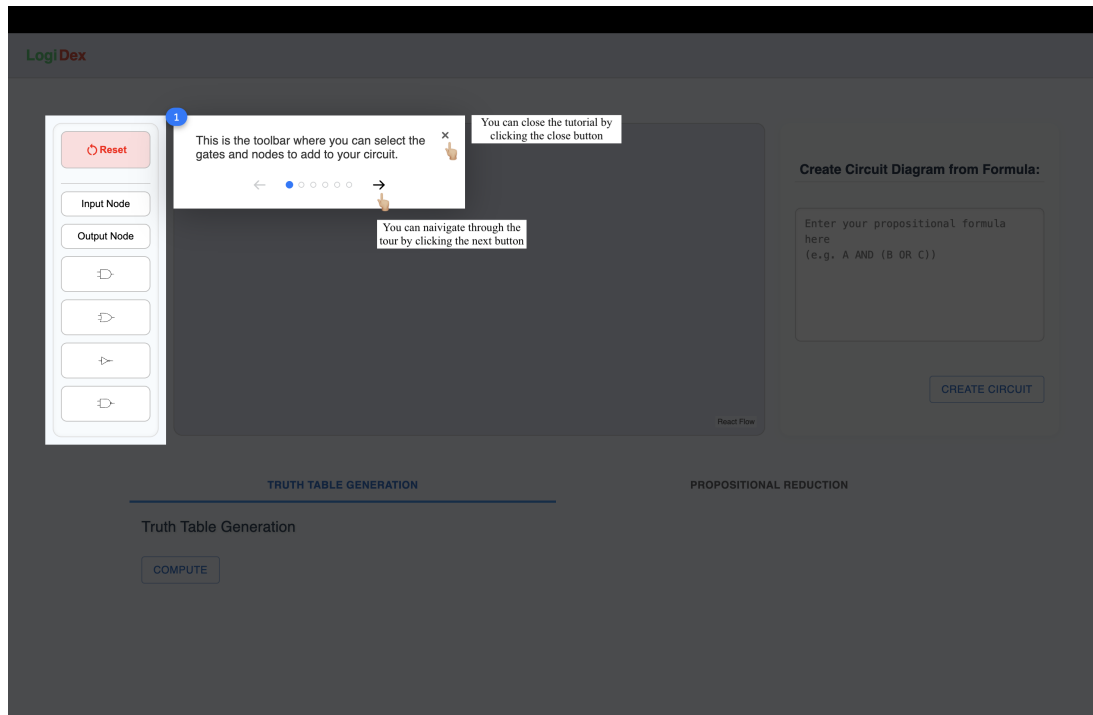
### Step 1: Landing Page and Tutorial Prompt

When the application is first loaded or refreshed, users are greeted with a welcome modal. This modal provides access to the tutorial for first-time users.



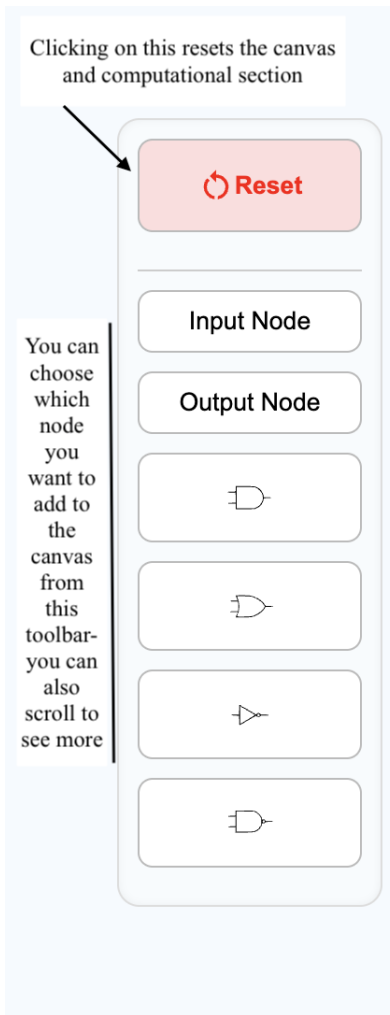
## Step 2: Navigating the Tutorial

The tutorial guides the user through the interface step by step. Use the navigation buttons to proceed through the tutorial or close it at any time.



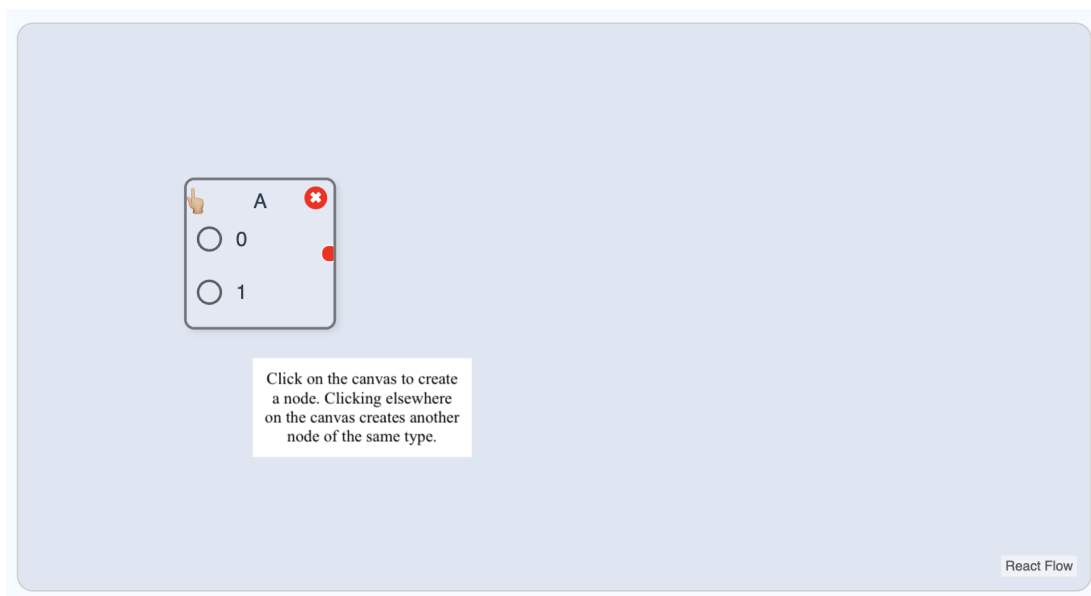
## Step 3: Exploring the Toolbar

The toolbar on the left provides access to logic gates, input/output nodes, and canvas controls like reset. Each icon is intuitive and positioned vertically for ease of access.



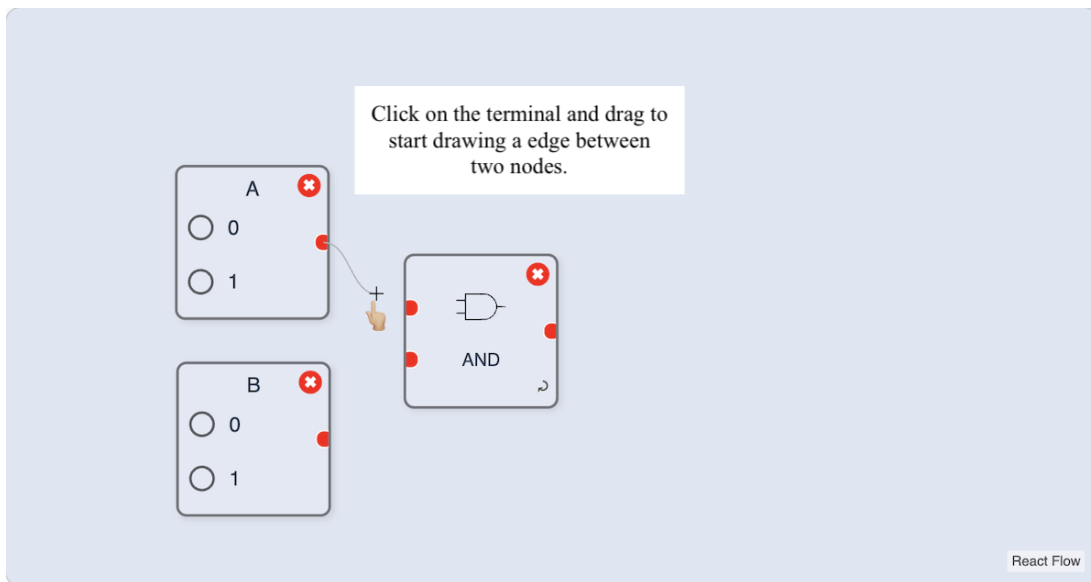
## Step 4: Placing Nodes on the Canvas

Clicking anywhere on the canvas will place the selected node at that location. This flexible interaction allows custom circuit layouts.



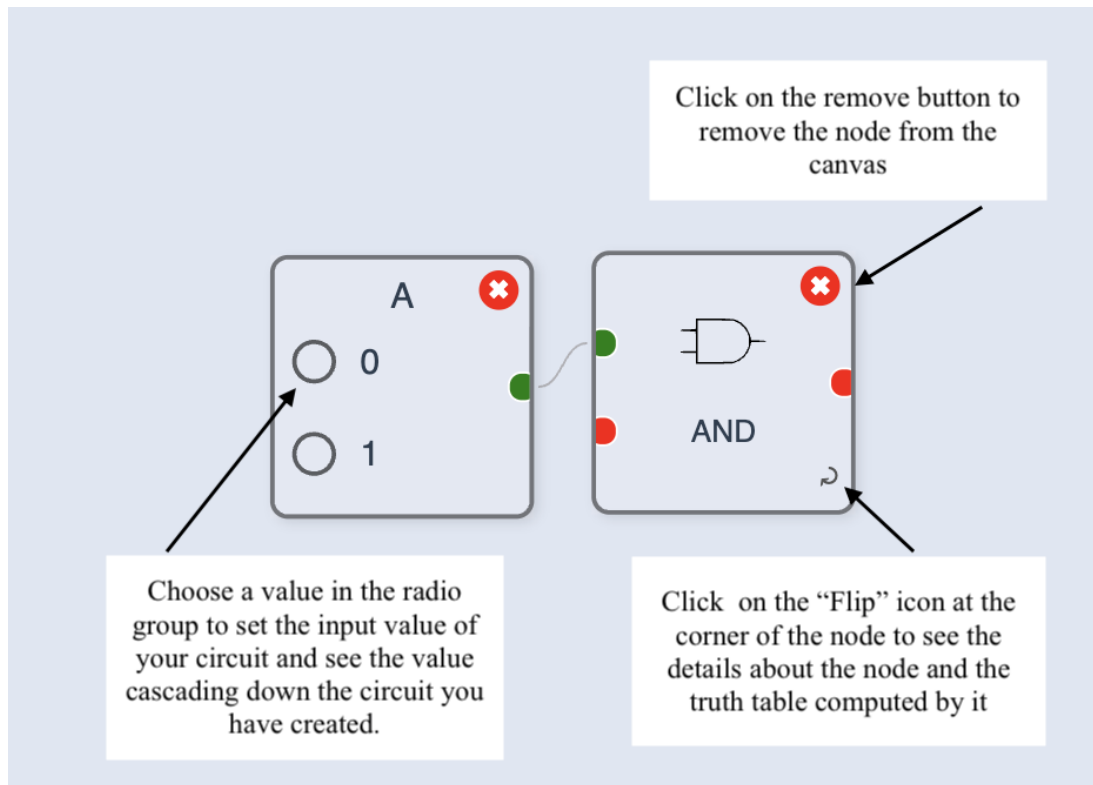
## Step 5: Drawing Connections Between Nodes

Click an output terminal (right side) and drag to an input terminal (left side) of another node to draw a wire. The cursor changes during the process for visual feedback.



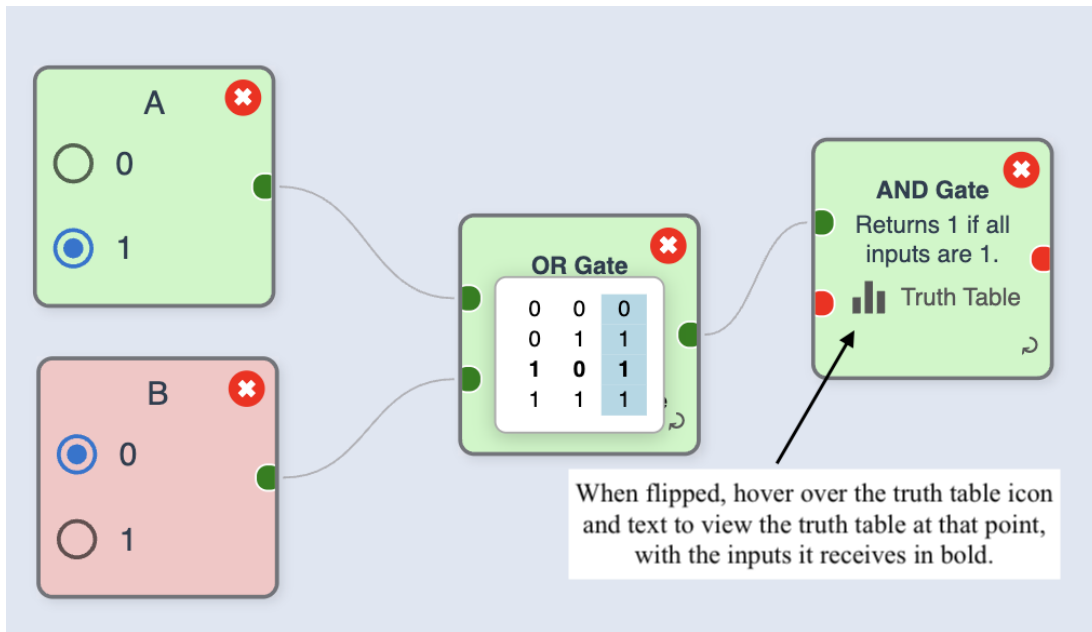
## Step 6: Node Options – Delete, Flip, and Input Control

Each node contains controls to delete it, flip its orientation, and for input nodes, select a value (0 or 1). These tools enable efficient circuit manipulation.



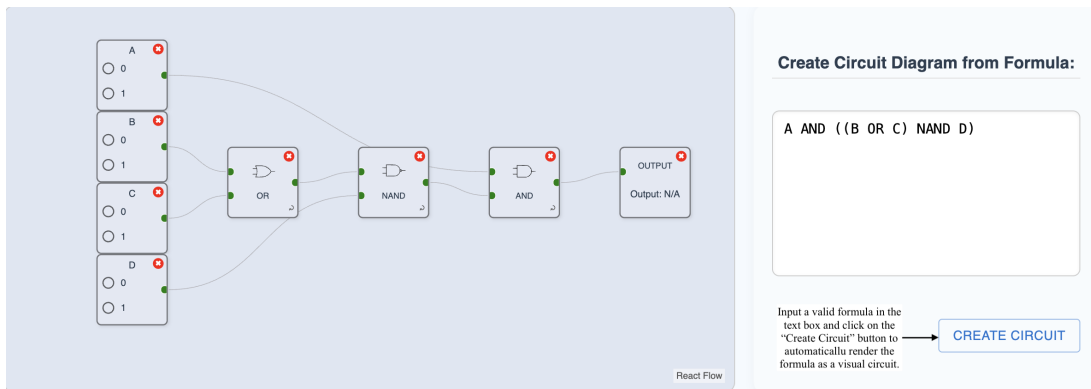
## Step 7: Flipping Nodes and Viewing Local Truth Tables

Nodes can be flipped for better alignment. Hovering over the "Truth Table" label reveals a small on-hover truth table for individual nodes.



## Step 8: Creating a Circuit from a Formula

Enter a propositional formula into the input box and click "Create Circuit." The input is validated and a matching circuit is generated visually.



## Step 9: Truth Table Generation and Filtering

Click "Compute" to generate the truth table for the circuit. Filters can be added or cleared to analyze specific logic paths.

### TRUTH TABLE GENERATION

**Truth Table Generation**

COMPUTE

Generated Truth Table

Rows Generated: **16/16** CLEAR ALL FILTERS

| A | B | C | D | Output |
|---|---|---|---|--------|
| 1 | 0 | 1 | 1 | 0      |
| 0 | 0 | 0 | 0 | 0      |
| 0 | 0 | 0 | 1 | 0      |
| 0 | 0 | 1 | 0 | 0      |
| 0 | 0 | 1 | 1 | 0      |
| 0 | 1 | 0 | 0 | 0      |

### PROPOSITIONAL REDUCTION

You can switch tabs here to view a different method of computation- propositional reduction and formula generation

Filters can be added here- rows can be filtered based on the value they have in that column

COMPUTE

Generated Propositional Formula

$(A \wedge \neg((B \vee C) \wedge D))$

Steps Generated: 4

CONJUNCTIVE NORMAL FORM
DISJUNCTIVE NORMAL FORM

**CNF Reduced Formula:**

$(A \wedge (\neg B \vee \neg D) \wedge (\neg C \vee \neg D))$

**Step 1**

**Apply De Morgan's Law to negated AND:**  $\neg((B \vee C) \wedge D) \rightarrow (\neg(B \vee C) \vee \neg D)$

Formula:  $(A \wedge (\neg(B \vee C) \vee \neg D))$

## Step 10: Propositional Reduction and CNF/DNF Tabs

Use the “Compute” button in the Propositional Reduction subtab to view the logical formula in symbolic form. Switch between CNF and DNF tabs for transformations.

### TRUTH TABLE GENERATION

### PROPOSITIONAL REDUCTION

**Propositional Reduction**

COMPUTE

Generated Propositional Formula

$(A \wedge \neg((B \vee C) \wedge D))$

Steps Generated: 4

CONJUNCTIVE NORMAL FORM
DISJUNCTIVE NORMAL FORM

**CNF Reduced Formula:**

$(A \wedge (\neg B \vee \neg D) \wedge (\neg C \vee \neg D))$

**Step 1**

**Apply De Morgan's Law to negated AND:**  $\neg((B \vee C) \wedge D) \rightarrow (\neg(B \vee C) \vee \neg D)$

Formula:  $(A \wedge (\neg(B \vee C) \vee \neg D))$

**Note:** All images include visual cues like boxes, arrows, and highlights to make the learning process intuitive and user-friendly.

## A.1 ReadME File: Instructions to Run the Code

### Overview

This is a web application for simulating and interacting with logic gates. It allows users to create circuits, visualize logic gate behavior, and interact with a propositional formula system.

### Requirements

- Python 3.x
- Node.js
- npm (Node Package Manager)

### Setup Instructions

Follow the steps below to set up and run the project.

#### 1. Backend Setup

1. Open a terminal window.
2. Navigate to the project directory.
3. Navigate to the **backend** folder:

```
cd backend
```

4. Create a virtual environment:

```
virtualenv venv
```

5. Activate the virtual environment:

- On macOS/Linux:

```
source venv/bin/activate
```

- On Windows:

```
.\venv\Scripts\activate
```

6. Install the required dependencies:

```
pip3 install -r requirements.txt
```

7. To run the backend server, execute:

```
python app.py
```

8. To run tests for the backend, use:

```
python run_tests.py
```

## 2. Frontend Setup

1. Open a new terminal window.
2. Navigate to the **frontend** folder:

```
cd frontend
```

3. Install the frontend dependencies:

```
npm install --legacy-peer-deps
```

4. Install the required versions of **ajv** and **ajv-keywords**:

```
npm install ajv@6.12.6 --legacy-peer-deps
```

```
npm install ajv-keywords@3.1.0 --legacy-peer-deps
```

5. To start the frontend application, execute:

```
npm start
```

### 3. Access the Application

- Once both the backend server and the frontend app are running, open your web browser (preferably Chrome).
- Navigate to `http://localhost:3000` to view the app and interact with the functionality.